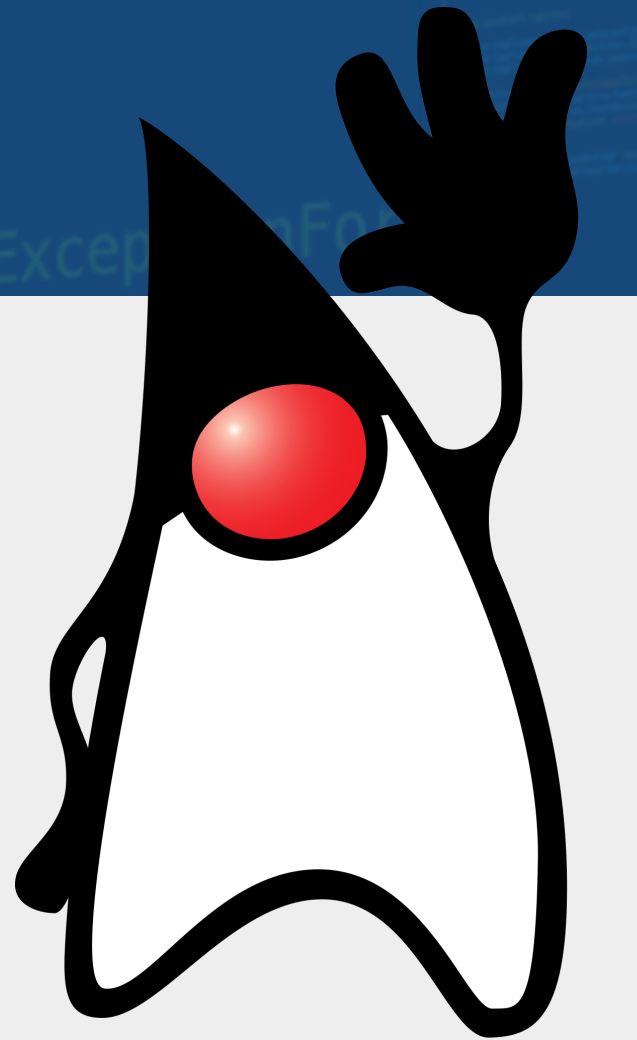


Java Programlama Dili

10. Bölüm Akış Kontrolü

Akın Kaldıroğlu



Amaç



- Bu bölümde akış kontrolü yapıları (flow control structures) ele alınacaktır
- Akış kontrolü yapıları ve bir sonraki diziler ile Java'nın yordamsal (procedural) kısmı tamamlanmış olacak
- Dolayısıyla yordamsal yöntemle gerçekleştirilebilecek algoritmik programlamanın temellerini bitirmiş olacağız
- Sonraki konular tamamen nesneler ilgili olacaktır



- **Akış Kontrolü**
- **Java'da Akış Kontrolü**
- **Tekrarlama**
 - while
 - do-while
 - for
 - for, while ve do-while
- **Karar Verme**
 - if Ailesi
 - Üçlü İşlemci ?:



- **Seçme**
 - switch Cümlesi
 - switch-case -> ile Cümle
 - switch-case -> ile İfade
 - switch-case yield ile İfade
 - switch-case Pattern Matching
- **Diğer Kontrol Yapıları**
 - break ve continue
 - return
 - Diğerleri
- **Özyineleme (Recursion)**
- **Erişilemeyen ve Ölü Kod**
- **Blok Kullanmak?**
- **Ödevler**
- **Bundan Sonra?**

Akış Kontrolü



Yapısal Programlama - I



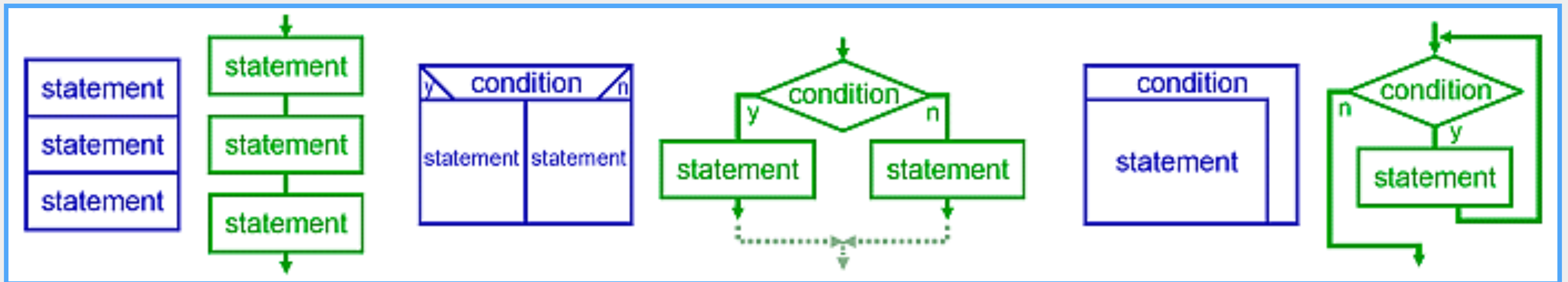
- **Yapısal Programlama (Structured Programming)**, 1960'larda temelde Algol diliyle geliştirilmiş bir programlama paradigmasıdır
- Amacı, *if then else*, *while*, *for* ve *subroutine* gibi kontrol yapılarının yaygın kullanımıyla programları bloklu (block-structured) hale getirerek, *spaghetti code*'dan kaçınmak ve kaliteyi arttırmaktır
- Sonrasında bu yaklaşım geliştirilmiş ve özellikle Algol-C-C++ geleneğini takip eden programlama dillerinin çekirdeğinde yer almıştır

Yapısal Programlama - II



- Corrado Böhm and Giuseppe Jacopini tarafından 1966'da yayınlanan bir makalede ifade edilen **Structured Programming Theorem**'e (ya da **Böhm–Jacopini Theorem**) göre aşağıdaki 3 temel kontrol yapısıyla herhangi bir fonksiyon (ya da algoritma) yazılabilir:
- **Art arda çalıştırma (concatenation)**
- **Tekrarlama, yineleme (repetition, iteration, looping)**
- **Karar verme, seçme (decision, selection)**

Yapısal Programlama - II



- https://en.wikipedia.org/wiki/Structured_program_theorem

Yapısal Programlama - IV

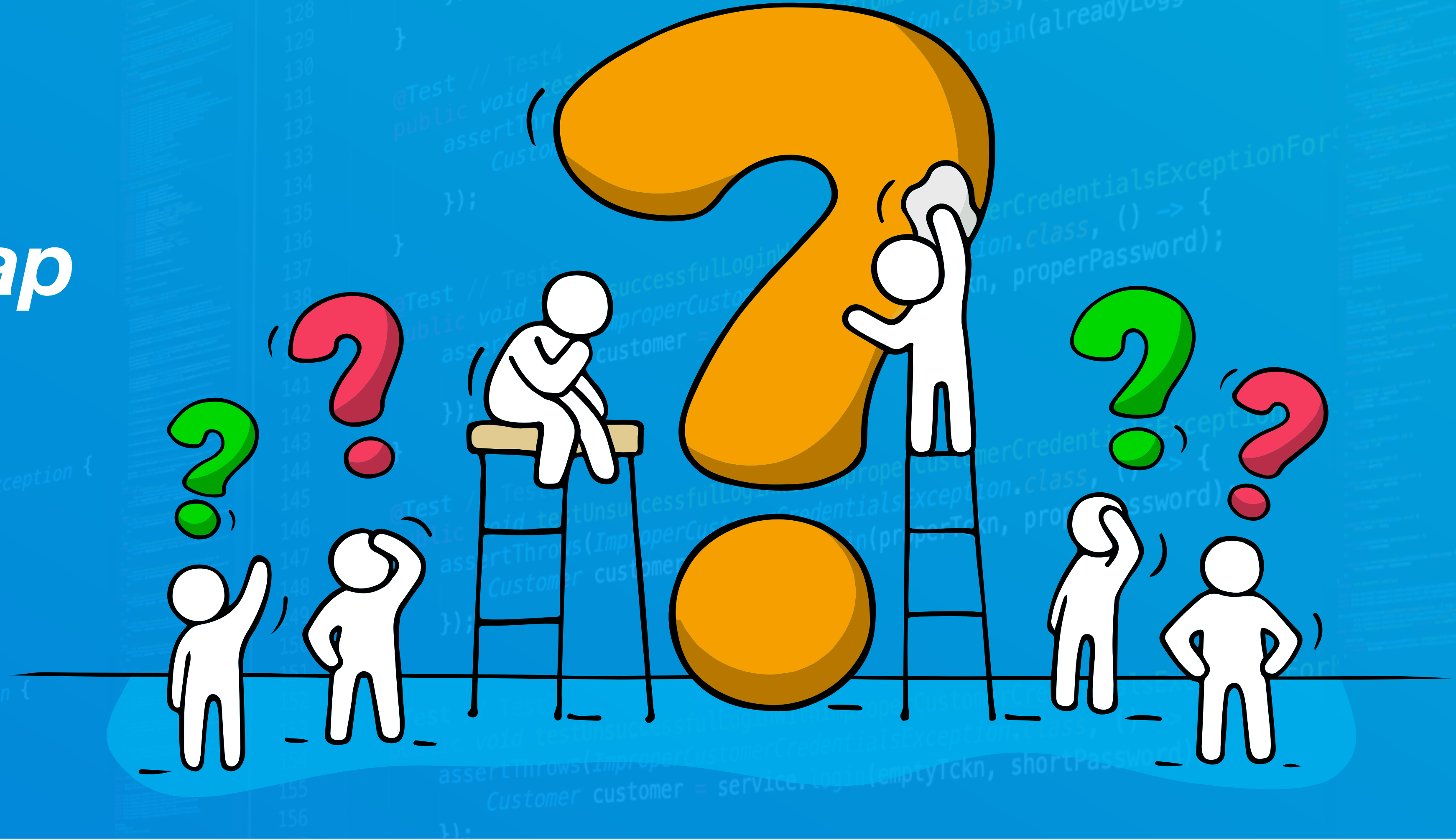


- Structured programming paradigmasına katkıda bulunanlardan birisi de Edsger W. Dijkstra'dır
- Dijkstra, ünlü *Goto Considered Harmful* makalesinde şöyle der:

The unbridled use of the go to statement has an immediate consequence that it becomes terribly hard to find a meaningful set of coordinates in which to describe the process progress

- http://www.u.arizona.edu/~rubinson/copyright_violations/Go_To_Considered_Harmful.html

Soru ve Cevap Zamanı!



Java'da Akış Kontrolü



Akış Kontrolü (Flow Control) - I



- Programlama dilleri, çalışma zamanında cümleleri tabi olarak **art arda (concatenation)** çalıştırırlar
- Bunun için cümleleri dilin yapısına uygun bir şekilde dizmek dışında bir şeye gerek yoktur
- Java'da bir şeyi **tekrar tekrar yapmak (repetition)** ya da **yinelemek (iteration)** için **while**, **do-while** ve **for** yapıları,
- **Karar vermek (decision)** için **if** ailesi, **seçmek (selection)** için ise **switch-case** yapıları vardır

Akış Kontrolü (Flow Control) - II



- Java'da bunlar dışında akışı yönlendiren, **break**, **continue**, **return** ve **try-catch-finally** yapıları vardır
- Ayrıca dilde kullanımı olmayan **goto** ayrılmış kelimesi de vardır
- Java'nın akış kontrolü yapıları C/C++'dakilerle hemen hemen aynıdır; sadece Java'da **goto**'nun bir kullanımı yoktur, bunun yerine etiketli **break** ve etiketli **continue** (labeled break ve labeled continue) vardır
- *Goto Considered Harmful*'u hatırlayın!

Kontrol Yapısında Parantez - I



- Java'nın karar verme ve tekrarlama yapılarında parantez () zorunludur
- Parantez sıklıkla mantıksal ifade için ya da `for`'da olduğu ilk değer atama gibi daha geniş amaçlarla kullanılır

```
while (mantıksal ifade)
    ...
```

```
do
    ...
while (mantıksal ifade)
```

```
for (başlangıç değeri; bitiş şartı; değişim)
    ...
```

```
if (mantıksal ifade)
    ...
else if (mantıksal ifade)
    ...
...
else if (mantıksal ifade)
    ...
else
    ...
```

Kontrol Yapısında Parantez - II



- Anahtar kelime ile parantez arasında boşluk(lar) olabilir
- Hatta Java kod geleneğinde yeri olmasa bile anahtar kelime ve parantez farklı satırlarda olabilir

```
while (mantıksal ifade)  
...
```

```
while  
    (mantıksal ifade)  
...
```

Cümle Ya da Blok



- Tüm kontrol yapılarında, aksi söylenmedikçe, tek bir cümle ya da sıfır veya daha çok cümle içeren bir blok kullanılabilir
- Yani aşağıdaki iki yapı da mümkündür, eğitim notlarında gösterim olarak hangisi uygunsa o kullanılacaktır
- Okunurluk söz konusu olduğunda bloklu şekil tercih edilmelidir

```
while (mantıksal ifade)  
    cümle
```

```
while (mantıksal ifade){  
    cümle(ler)  
}
```

Kod Geleneğinde Blok



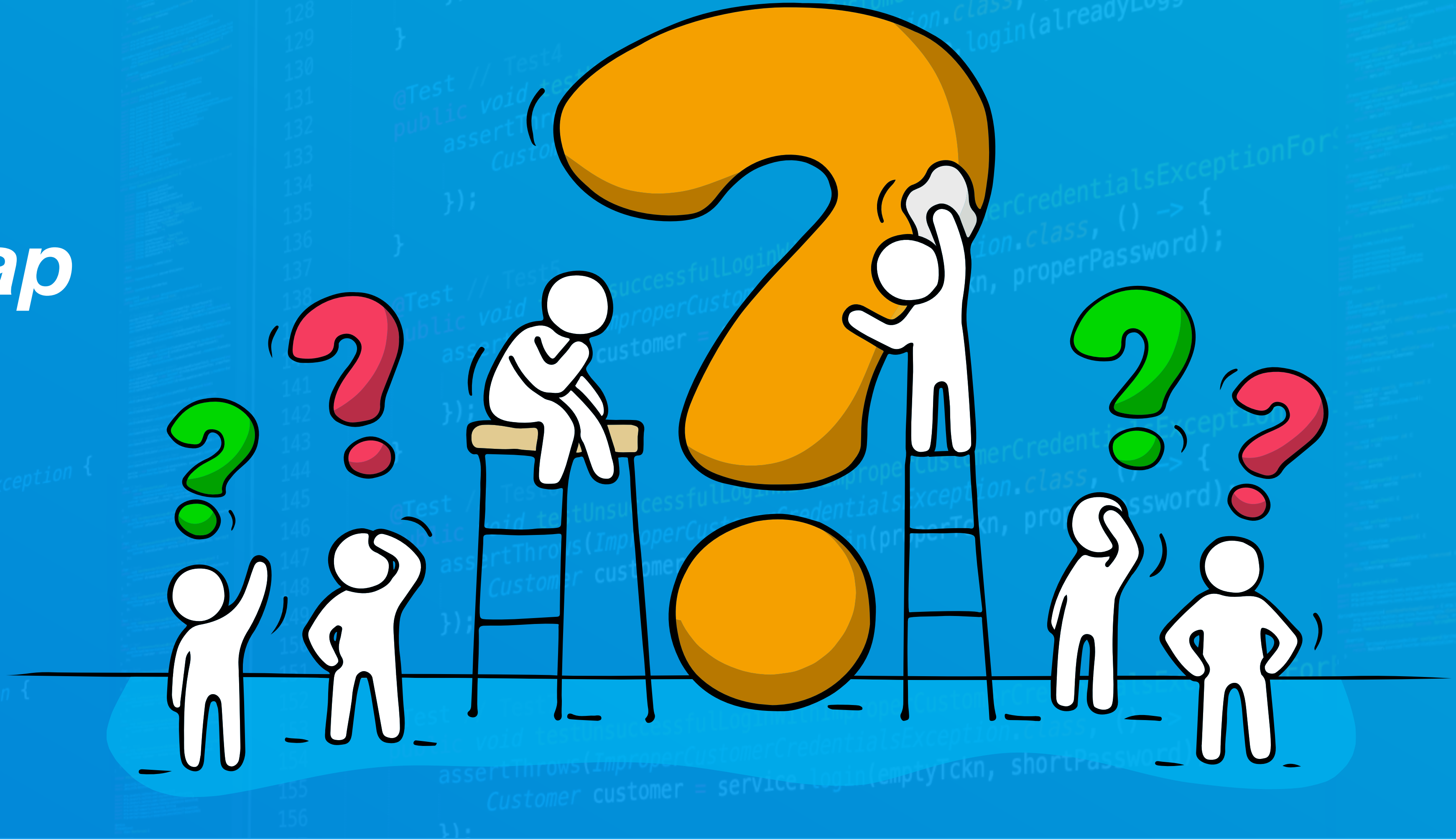
- Ayrıca Java kod geleneğinde blok başlangıcı, kontrol yapısıyla aynı satırda olur

```
for (...){  
    cümle(ler)  
}
```

```
while (mantıksal ifade){  
    cümle(ler)  
}
```

```
do  
    ...  
while (mantıksal ifade){  
    cümle(ler)  
}
```

Soru ve Cevap Zamanı!



Tekrarlama



Tekrarlama - I



- Java'da bir şeyi **tekrar tekrar yapmak (repetition)**, **yinelemek (iteration)** için aşağıdaki yapılar vardır:
 - **while**
 - **do-while**
 - **for**
- Bu yapılara **döngü (loop)** de denir

Tekrarlama - II



- `while` ve `do-while` çok benzerdir fakat `for` biraz farklıdır ve kontrolü daha kolaydır

```
while (mantıksal ifade)  
    cümle
```

```
do  
    cümle  
while (mantıksal ifade)
```

```
for (başlangıç değeri; bitiş şartı; değişim)  
    cümle
```

- Şimdi sırayla bu yapıları ele alalım



while

while - I



- **while**, kendisinden sonra gelen mantıksal (boolean) ifade doğru yani **true** değerine sahip olduğu müddetçe, kendisine bağlı olan cümle ya da bloğun tekrar tekrar çalışmasını sağlar
- Mantıksal ifade **false** olduğunda **while** biter ve akış **while**'dan sonraki satırdan devam eder

```
while (mantıksal ifade)  
    cümle
```

```
while (mantıksal ifade) {  
    cümle(ler)  
}
```

WhileDemo



- `loop.WhileDemo`

while - II



- **while**'in mantıksal ifadesi bileşik ifade (compound expression) de olabilir
- Önemli olan ifadenin **true** ya da **false** bir sonuç vermesidir

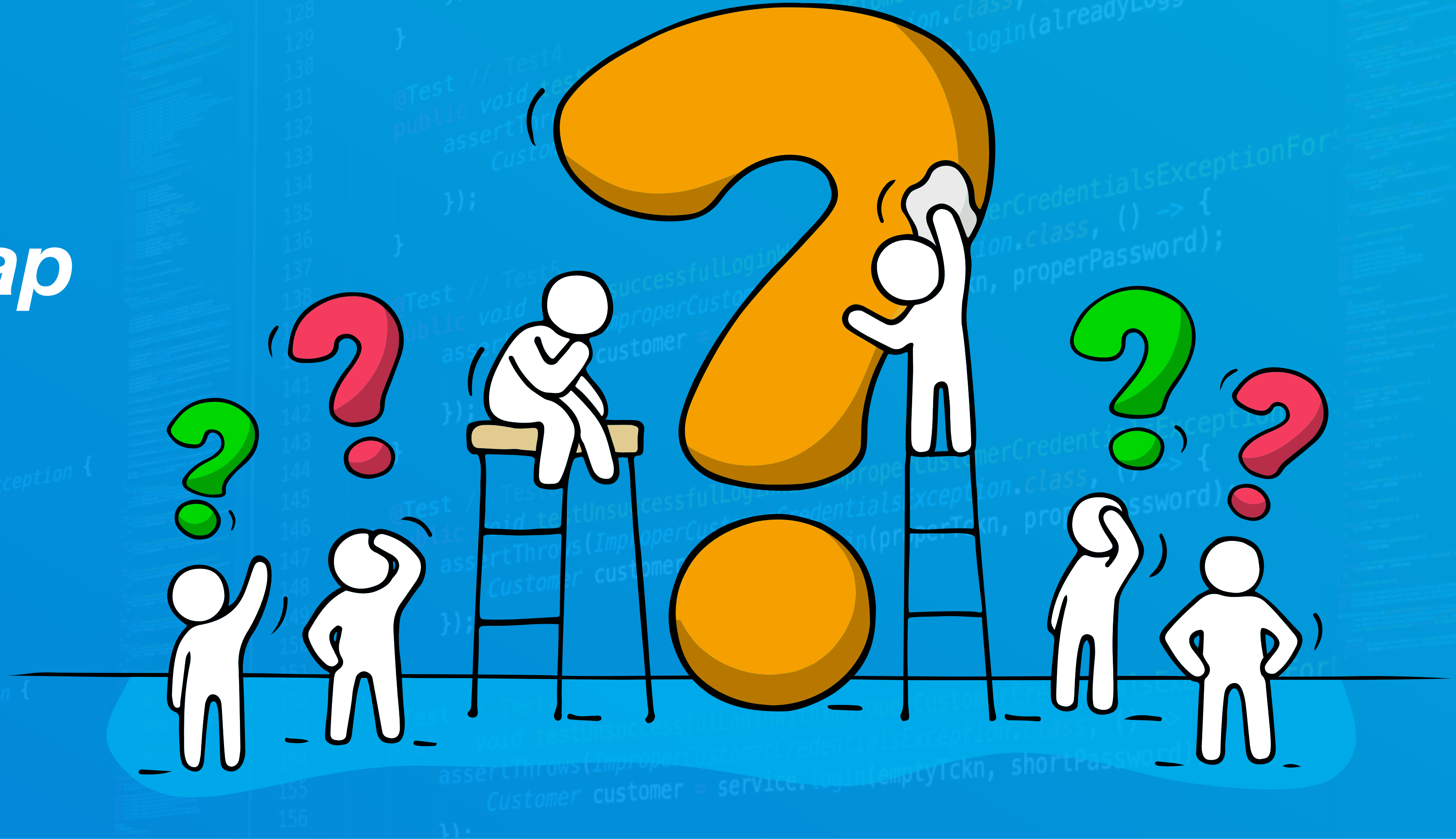
```
double i = 12.9;
double j = 7.4;
boolean b;
while (b = i > j ? true : false) { // No need for ?:, i > j just is enough
    System.out.println("i: " + i + " j: " + j);
    i = Math.random();
    j = Math.random();
}
```

ComplexWhile



- `loop.ComplexWhile`

Soru ve Cevap Zamanı!





do while

do while - I



- **do while**, **while**'in kardeşi bir yapıdır
- **do while** döngüsünde **do**'dan sonra gelen cümle ya da cümle bloğu, **while**'in mantıksal ifadesi doğru olduğu müddetçe tekrar tekrar çalıştırılır
- Mantıksal ifade **false** olduğunda **do while** döngüsü biter ve akış **do while**'dan sonraki satırdan devam eder

do while - II



- **do while**'in **while**'dan farkı, **do**'dan sonraki cümle ya da bloğun en az bir kere çalışmasıdır, çünkü **while** içindeki şart kontrolü sonda yapılmaktadır
- **while** satırındaki **;**'ü unutmayın

```
do  
    cümle  
while (mantıksal ifade);
```

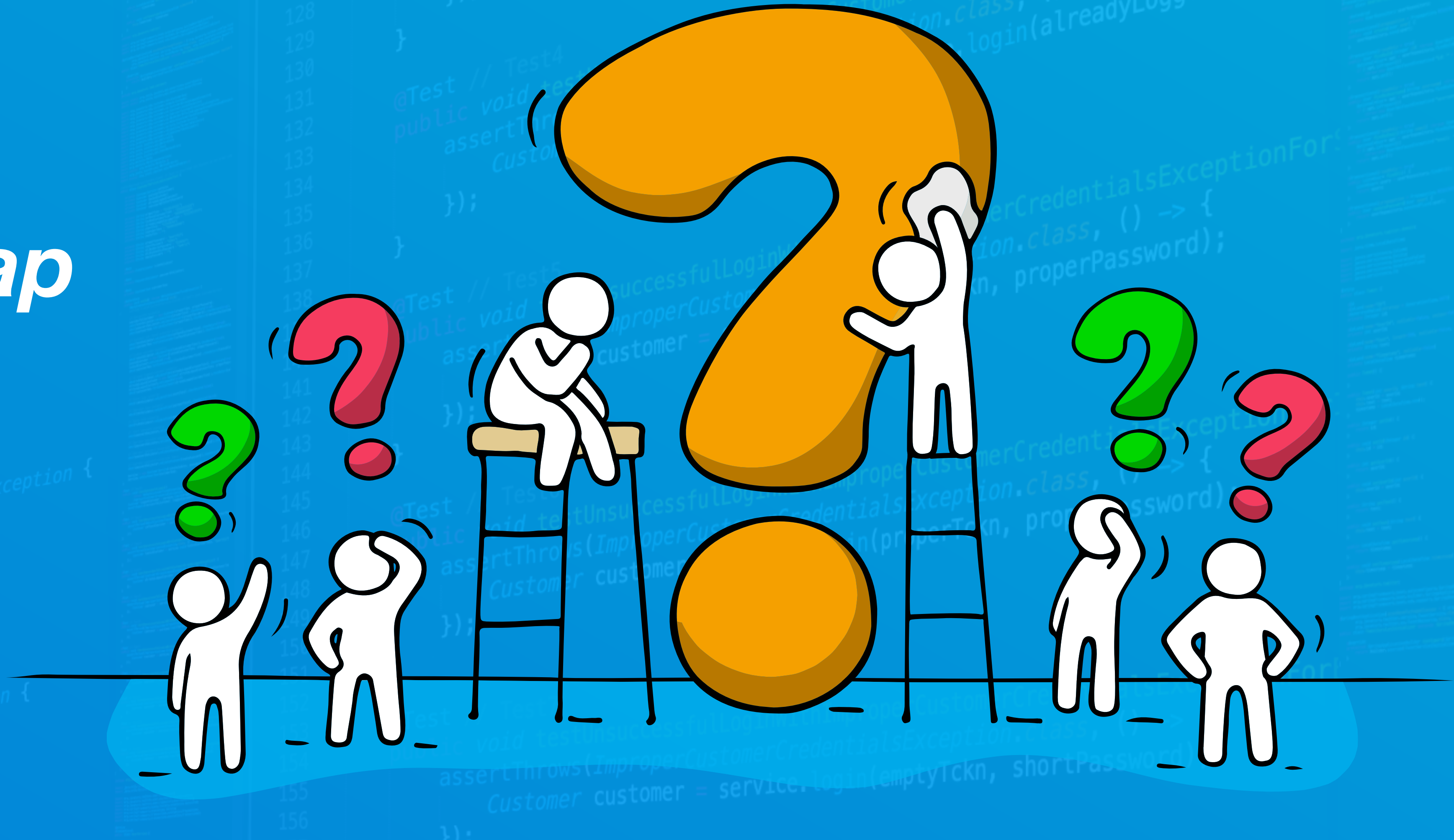
```
do{  
    cümle(ler)  
}  
while (mantıksal ifade);
```

DoWhileDemo



- `loop.DoWhileDemo`

Soru ve Cevap Zamanı!





for

for - I



- **for**, sayaç kullanarak bir cümle ya da bloğu tekrarlı olarak çalıştırır
- Cümle ya da bloğun kaç defa çalıştırılacağı, sayacın başlangıç değeri, bitiş şartı ve değişim bilgisi ile belirlenir
- Başlangıç değeri, bitiş şartı ve değişim zorunlu değildir ama parantez zorunludur ve içinde iki tane **;** bulunmalıdır

```
for (başlangıç değeri; bitiş şartı; değişim)  
    cümle
```

```
for (başlangıç değeri; bitiş şartı; değişim){  
    cümle(ler)  
}
```

for - II



- **for**, önce sayaç için başlangıç değerini belirler sonra sayacın bitiş şartını kontrol eder ve bitiş şartı gerçekleşmemişse, cümleyi ya da bloğu çalıştırır ve son olarak da sayacın değerini değiştirir
- Çalışma, değişimle elde edilen yeni sayaç değerini, bitiş şartıyla karşılaştırarak devam eder
- Bitiş şartı gerçekleşmemişse cümle/blok çalıştırılır, gerçekleştiğinde ise tekrarlama biter

```
for (başlangıç değeri; bitiş şartı; değişim){  
    cümle(ler)  
}
```

for - III



- **for** döngüleri sıklıkla iç içe kullanılır:
 - Çok boyutlu dizilerde, matematiksel işlemlerde ya da pek çok iş kuralında 2 ya da 3 tane **for** döngüsü iç içe olur
- **for** döngülerindeki sayaçlar sıklıkla **i**, **j**, **k** gibi isimler alır
- İlk etapta kötü görünse bile bu değişkenlerle sayaç tutulduğu anlaşılır ve bir anlama problemi oluşmaz
- **i**, **j**, **k**'nın Fortran'dan başlayan bir gelenek olduğunu tahmin ediyorum

ForDemo - I



- `loop.ForDemo`
- `simpleFor()`
- `multiplicationTable()`

ListCharacters



- `loop.ListCharacters`

for - IV



- **for**'da birden fazla sayaç değişkeni kullanılabilir
- Eğer değişkenler **for** içinde tanımlanıyorsa değişkenlerin tipleri aynı olmalıdır aksi takdirde farklı tiplerden olabilir
- Bitiş şartı bileşik bir ifade olabilir
- Değişimde birden fazla değişken güncellenebilir

```
for(int i = 1, j = i + 10; i < 5 && j > 2; i++, j = 2*i)
    System.out.println("i= " + i + " j= " + j);

//      That's error!
for(int i = 1, long j = i + 10; i < 5 && j > 2; i++, j = 2*i)
    System.out.println("i= " + i + " j= " + j);
```

ForDemo - II



- `loop.ForDemo`
- `others()`

FibonacciNumbers



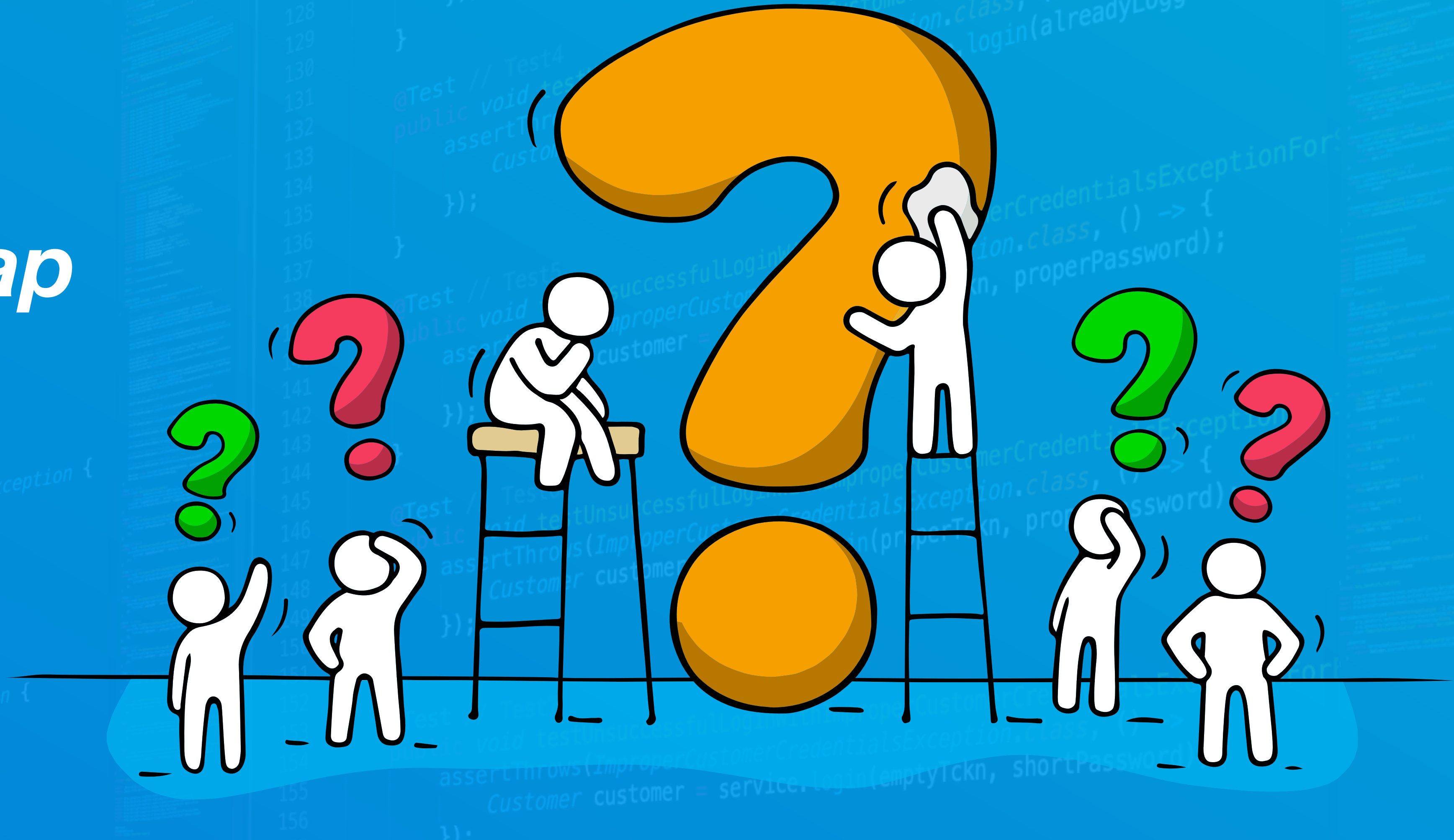
- `loop.FibonacciNumbers`

For Each



- **for** yapısının genelde **for each** olarak adlandırılan ve torbalar (collections, containers) üzerinde çalışan bir gelişmiş hali daha vardır
- Bu yapı ileride ele alınacaktır

Soru ve Cevap Zamanı!





for, while, ve do while

for, while ve do-while



- Pek çok problem hem `for` hem de `while` ve `do while` ile çözülebilir
- Ama yine de problemler genelde birisiyle çözülmeye daha uygundur:
 - `for` daha çok tekrar sayısının önceden bilinen faktörlerle belirlendiği durumlara uyarken `while` ve `do while`, tekrar sayısının, tekrarlar sırasında ortaya çıkan durumlara bağlı olduğu hallere uygun düşmektedir
- Bu sebeple `for` için *belirlenmiş döngü* anlamında *determinate loop* da denir

ForAndWhile



- `loop.ForAndWhile`

Durma Problemi - I



- Aşağıdaki kodlarda problemi olan şey nedir?

```
while (true)
    System.out.println("");
```

```
double d = Math.random();
for ( ; ; )
    System.out.println(d);
```

```
double d = Math.random();
while (d < 0.5)
    System.out.println(d);
```

- Bu örneklerde **durma problemi (halting problem)** vardır
- Kod ilk iki örnekte kesin bir şekilde son örnekte de 0.5'ten küçük olan herhangi bir değer için sonsuz döngüye girecektir

Durma Problemi - II



- Kodda bir **durma problemi (halting problem)** vardır
- **while** veya **for** gibi tekrar yapılarının sonsuz döngüye girmemesi için gerekli önlemler alınmalıdır
- Örneğin tekrar yapısının içinde değer yenilenebilir ya da durmayı sağlayacak bir yöntem geliştirilebilir

```
double d = Math.random();
while (d < 0.5){
    System.out.println(d);
    d = Math.random();
}
```

```
int counter = 0;
double d = Math.random();
while (d < 0.5){
    System.out.println(d);
    counter++;
    if(counter > 100) break;
}
```

HaltingProblem1



- `loop.HaltingProblem1`

Durma Problemi - III



- Ayrıca döngülerde kıyaslama işlemcileri özellikle de eşitlik kıyaslaması kullanılırken kayan noktalı (floating point) değer kullanılması durumunda hiçbir zaman durmayacak döngülere girilebilir; dikkatli olmak gerekir

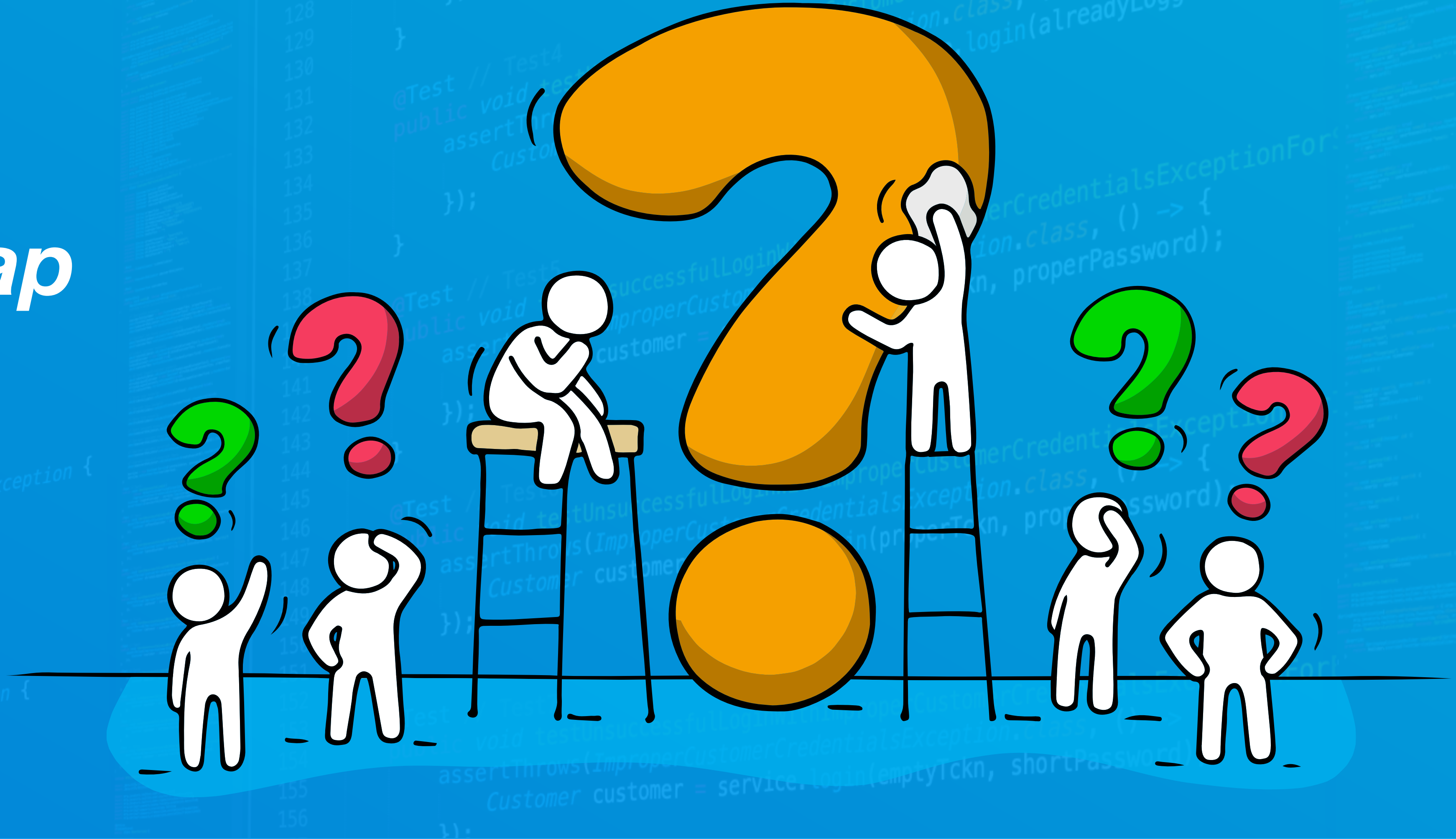
```
for(double d = 0.0; d  $\neq$  1.0; d = d + 0.1)  
...
```

HaltingProblem2



- `loop.HaltingProblem2`

Soru ve Cevap Zamanı!



Karar Verme



Karar Yapıları (Decision Structures)



- Java'daki **karar yapıları** (decision structures) şunlardır:
 - **if** ailesi:
 - **if**
 - **if else**
 - **if else ...**
 - Üçlü işlemci (ternary operator) **?:**
 - **switch case** ailesi (daha çok seçme amacıyla kullanılır)



if Ailesi



- **if** mantıksal bir ifadeye bağlı olarak karar vermek için kullanılır
- **if** daima mantıksal bir ifade alır, doğru ise bağlı cümle ya da blok çalışır, yanlışsa çalışmaz
- Her iki halde de akış, **if** cümlesi ya da bloğundan sonraki satırdan devam eder

```
if (mantıksal ifade)  
    cümle
```

```
if (mantıksal ifade){  
    cümle(ler)  
}
```

- Okunurken *if then* denir ama Java'da *then* anahtar kelimesi yoktur

ListCharactersWithIf



- `decision.OrElse.ListCharactersWithIf`

if-else



- **if**'in mantıksal ifadesi doğru olmadığında çalışacak alternatif bir durum var ise **else** kullanılır

```
if (mantıksal ifade)
    cümle
else
    alternatifCümle
```

```
if (mantıksal ifade){
    cümle(ler)
}
else {
    alternatifCümle(ler)
}
```

Sarkan else - I



- Bir **if**'in içinde başka bir **if** varsa, **else**'in hangisine ait olduğu sorusu ortaya çıkar

```
if (i > j)
    if (i - j > 10)
        System.out.println("i is at least 10 greater than j.");
else
    System.out.println("i is not greater than j."); // but it connects to the one inside!
```

- Bu duruma **sarkan else (dangling else)** denir

Sarkan else - II



- **else**, kendisinden önce gelen ve henüz bir **else** ile eşleşmemiş en yakın **if**'e bağlanır
- Girinti (indentation) derleyici için hiçbir anlam taşımaz; sadece okuyanı yanıltır
- IDE'lerin şekil verme (formatting) özellikleri bu durumun önüne geçmeye çalışır

```
if (i > j)
    if (i - j > 10)
        System.out.println("i is at least 10 greater than j.");
else
    System.out.println("i is not greater than j."); // but it connects to the one inside!
```

DanglingElse



- `decision.ifElse.DanglingElse`

else if - I



- Java'da **else if**, diye bir yapı yoktur
- Daha önce bahsetmiştik **else**'ten sonra bir cümle (statement) gelir
- **if** bir cümledir, **while** bir cümledir, **for** bir cümledir
- Dolayısıyla **else if** gördüğümüz şey, aslında **else** ve ardından gelen bir **if** cümlesidir

```
int i = 10;  
  
if (false) { ... }  
else if (i > 0) { ... } // else + if statement
```

```
int i = 10;  
  
if (false) { ... }  
else  
    if (i > 0) { ... } // better
```

else if - II



- Aynı sebeple **else while** ve **else for** da geçerlidir
- Geçerli olması, yazılması gerektiği anlamına gelmez; okunurluk açısından tercih edilmez

```
int i = 10;

if (false) { ... }
else if (i > 0) { ... }           // else + if statement

if (false) { ... }
else while (i > 0) { ... }       // else + while statement

if (false) { ... }
else for (; i > 0; i--) { ... }  // else + for statement
if (i > j)
```

IfElseWhile



- `decision.ifElse.IfElseWhile`

if else-if else



- Birden fazla şarta bağlı olarak çalışacak alternatif cümleler ya da bloklar var ise **if else if else** ile yapılır
- Bu yapıdaki **else if**, alternatif sayısı kadar artabilir
- Sondaki **else** zorunlu değildir ama varsa sadece bir tane ve en sonda olabilir

```
if (mantıksal ifade)
    cümle
else if (mantıksal ifade)
    alternatifCümle
...
else if (mantıksal ifade)
    alternatifCümle
else
    alternatifCümle
```

IfElseDemo



- `decision.ifElse.IfElseDemo`

== ile Eşitlik Kıyaslama (Tekrar) - I



- Unutmayın, eşitlik kıyaslaması için ==, eşitsizlik kıyaslaması için ise != kullanılır
- `if` ya da `while` gibi yapılarda yanlışlıkla eşitlik kıyaslaması için == yerine atama işlemcisi = yazmak yaygın bir hatadır

== ile Eşitlik Kıyaslama (Tekrar) - II



- `if` ya da `while` daima mantıksal bir ifade alır
- Atama ifadesinin değeri, atama yapılan değişkenin değeridir
- Yanlışlıkla değer atanan değişken `boolean` değilse derleyici hata verir,
- Aksi taktirde `boolean` olan bir değişkene atama yapılırsa derleyici hata vermez ve beklenmeyen sonuçlar oluşabilir

```
int i;  
"  
if (i = 10) //Compiler error!  
    cümle1  
else  
    cümle2
```

```
boolean b = false;  
if (b = true) // = olmalıydı!  
    cümle1 // Gerçekte seçilen  
else  
    cümle2 // = olsaydı seçilecek olan
```

IfWithAssignment



- `decision.ifElse.IfWithAssignment`



- Aşağıdaki sınıflandırmaya göre girilen bir yaşın hangi aralıkta olduğunu bulan ve yazan bir program yazın
- < 0 ya da > 120 : Çık
- 0-3 : Bebek
- 4-12 : Çocuk
- 13-19 : Ergen
- 20-30 : Genç
- 31-59 : Orta Yaş
- 60-120 : Yaşlı



- Ayrıca yukarıdaki sınıflandırmaya göre bir ailedeki bireylerin yaşlarını alıp, aralığını veren ve en sonunda da ailede hangi aralıktan kaç kişi olduğunu yazan bir program yazın



- * kullanarak verilen bilgilerle bir sonraki sayfadaki şekilleri çizen programları, **while**, **do-while** ve **for** kullanarak yazın:
- Yükseklik ile dik üçgen,
- Yükseklik ile eşkenar üçgen ve baklava dilimi (diamond)
- Yükseklik ve genişlik ile dikdörtgen
- Şekilleri içleri dolu ve boş olarak çizebilirsiniz



```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

```
*****
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*                               *
*****
```

```
*
***
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

```
      *
     ***
    *****
   ********
  *********
 ***
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

Cümle Problemi - I



- Aşağıdaki gibi bir durumla karşılaştınız mı?

```
if(true)
    int u = 7;  // Compiler error!

for (int j = 0; j < 10; j++)
    int k = j;  // Compiler error!
```

Cümle Problemi - II



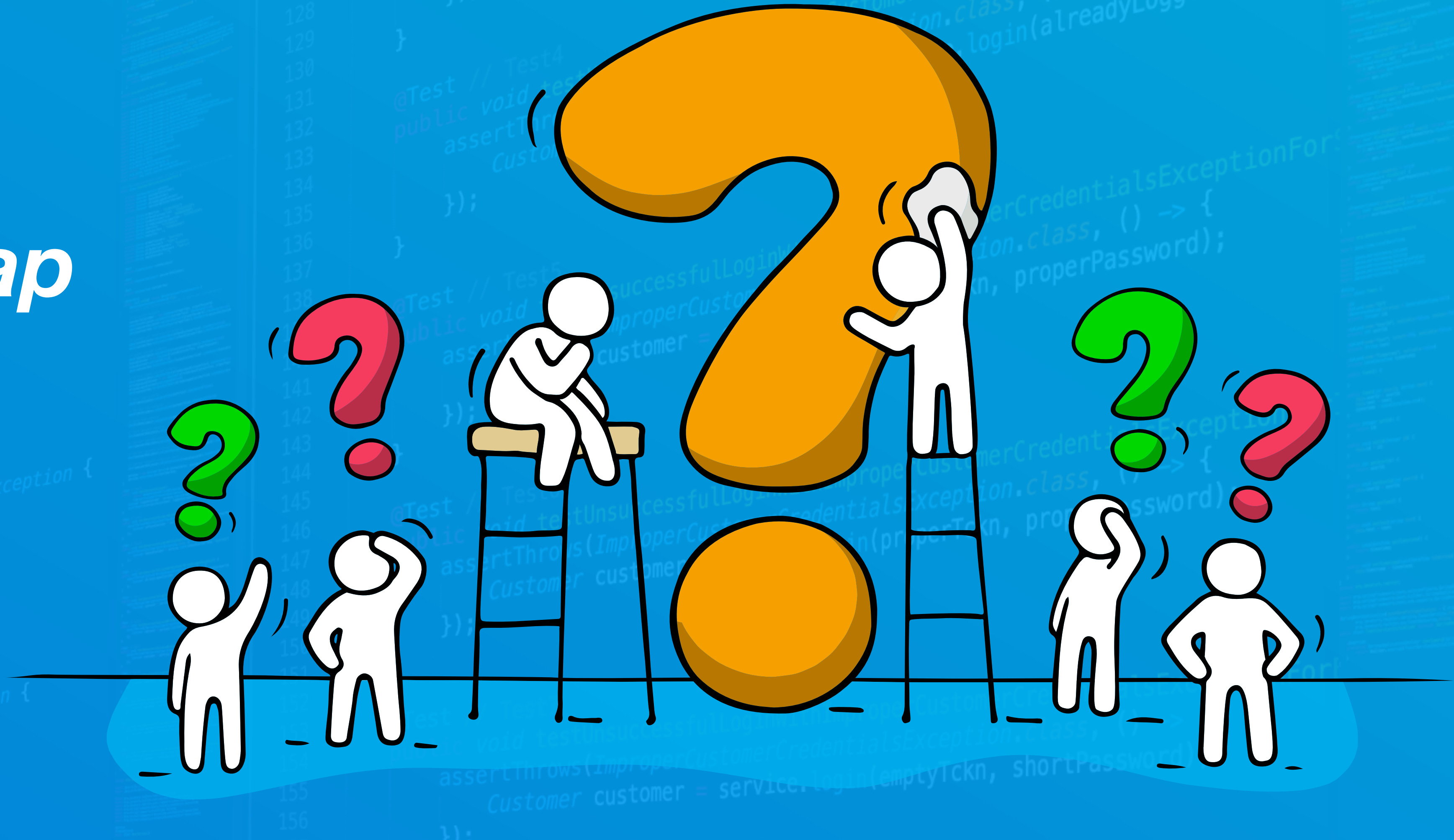
- Problem, blok kullanılmadığında, `if` ve bir değişken tanımının tek bir cümlede (statement) olması ve değişkene bir kapsam sağlanmamasıdır
- Aslında değişken tanımı zaten tek bir cümledir ve bu şekilde bir `if` (ya da `for`, `while` ve `do-while`) cümlesinin içinde kullanılamaz
- Bu yüzden Java derleyicisi “*variable declaration not allowed here*” der
- Çözüm, blok kullanmak veya değişkeni önceden tanıtmaktır

StatementProblem



- `StatementProblem`

Soru ve Cevap Zamanı!





Üçlü İşlemci ?:

Üçlü Şart İşlemcisi - I



- Hatırlayın, **üçlü şart işlemcisi** (**ternary conditional operator**), **?:** üç tane işlenen alan tek işlemciydi
- Üçlü işlemci, **if-else** cümlesinin kısaltılmış halidir
- İlk ifade **boolean** olmalıdır ve doğru ise ikinci ifade, değil ise üçüncü ifade, en soldaki değere atanır

```
variable = mantıksal ifade ? ifade1 : ifade2
```

Üçlü Şart İşlemcisi - II



```
variable = mantıksal ifade ? ifade1 : ifade2
```

```
if (mantıksal ifade)
    variable = ifade1;
else
    variable = ifade2;
```

```
double random = Math.random();
String para = random > 0.5 ? "Tura" : "Yazı";
```

```
String para;
double random = Math.random();
if(random > 0.5)
    para = "Tura";
else
    para = "Yazı";
```

Üçlü Şart İşlemcisi - III



- Üçlü şart işlemcisi herkesin tercih ettiği bir işlemci değildir
- Pek çok geliştirici, `if-else` yapısını kullanmayı, okunurluk açısından ve çoğunlukla da alışkanlıkla üçlü şart işlemcisine tercih eder
- Özellikle de üçlü şart işlemcisinin iç içe kullanımı söz konusuysa, bu durumda `if-else` yapısı, okunurluktan dolayı tercih edilmelidir
- Soldaki kod, değerlendirme sırasından dolayı sağdaki gibi çalışır

```
b = x < y ? x < z ? true : false : false;
```



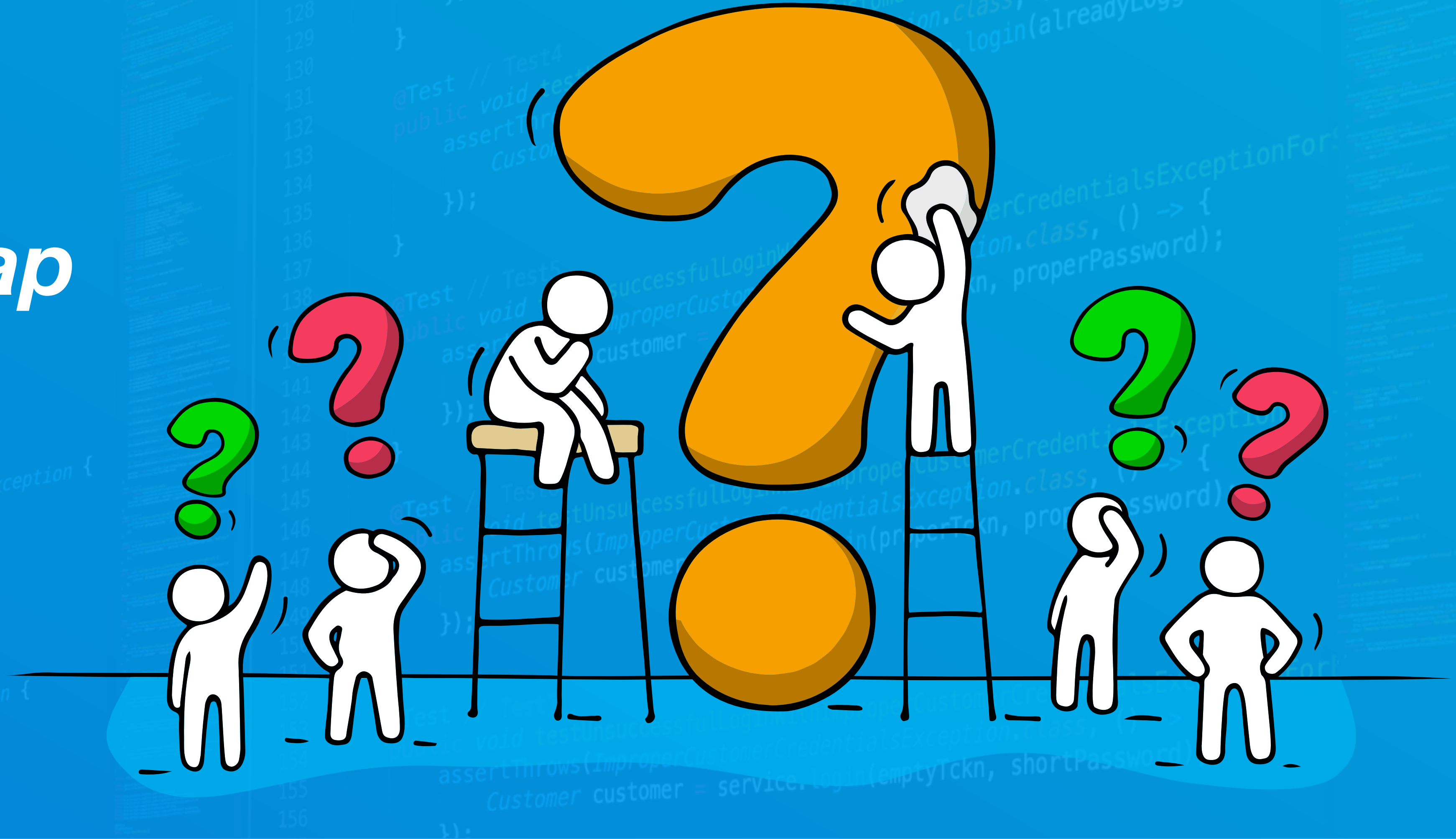
```
b = x < y ? (x < z ? true : false) : false;
```

TernaryOperatorDemo



- `decision.TernaryOperatorDemo`

Soru ve Cevap Zamanı!



Secme



switch Ailesi



- **switch**, birazdan detaylıca öğreneceğimiz yapısından dolayı karar vermekten çok seçme amacıyla kullanılır
- Her seçme bir karar içerir tabii olarak
- Java'nın ilk sürümünde gelen **switch** cümlesi ileriki sürümlerde farklı şekillerle genişledi ve bir aile halini aldı
- Şimdi bu aileyi görelim



switch Cümlesi

switch Cümlesi - I



- **switch** cümlesinin C'den devralındığı şekliyle, Java'nın ilk sürümünden bu yana var olan halidir
- **switch**'e geçilen `int` ya da `String` değerindeki *seçici ifade* (*selector expression*), hangi **case** sabitesine eşitse ondan sonra gelen ifadeler ilk **break**'e kadar çalıştırılır

```
switch(int/String-ifade){  
    case sabite1:  cümle(ler); break;  
    case sabite2:  cümle(ler); break;  
    . . .  
    default:      cümle(ler); break;  
}
```

switch-case - II



- **switch-case** if else if else'in int ve String tipinden, sadece tekil değerler alan ifadeler için daha toplu bir şeklidir
- **case**'den sonra gelen değer **sabite (literal)** ya da **sabit değişken (constant variable)** olmalıdır ve sonrasında **:** kullanılır
- **:**'den dolayı **switch-case**'in bu haline *Colon Cases* de denir

```
switch(int/String-ifade){  
    case sabite1:  cümle(ler); break;  
    case sabite2:  cümle(ler); break;  
    . . .  
    default:  cümle(ler); break;  
}
```

SimpleSwitchStatement



- `decision.switchCase.SimpleSwitchStatement`

switch-case - III



- **switch**'e geçilen *int* ya da *String* ifade, hangi **case** sabitesine eşitse ondan sonra gelen ifadeler **break**'e kadar çalıştırılır dendi
- Eğer hiçbirine eşit değilse, varsa **default**'tan sonrası **break**'e kadar çalıştırılır, yoksa **switch** bloğundan çıkılır
- Karşılaşılan ilk **break**'ten sonra akış **switch** blokundan çıkar ve sonraki ilk cümle çalıştırılır

```
switch(int/String-ifade){  
    case sabite1:  cümle(ler); break;  
    case sabite2:  cümle(ler); break;  
    . . .  
    default:      cümle(ler); break;  
}
```

switch-case - IV



- **case** ya da **default**'tan sonra sıfır ya da daha fazla cümle olabilir; bu cümleler bir blok içinde yer alabilir

```
switch(int/String-ifade){  
    case sabite1:  cümle(ler); break;  
    case sabite2:  cümle(ler); break;  
    . . .  
    default:  cümle(ler); break;  
}
```

```
switch(int/String-ifade){  
    case sabite1:  {cümle(ler); break;}  
    . . .  
    default:  {cümle(ler); break;}  
}
```

- **switch** bir bloktur, içi tamamen boş olabilir ama muhakkak bir seçici ifade almalıdır

```
switch(int/String-ifade){  
    // Empty block!  
}
```

switch-case - V



- Java 14'e kadar **case**'den sonra sadece tek bir değer kullanılabılırdı, Java 14 ile birlikte **case**'den sonra virgülle listelenen birden fazla değer kullanmak mümkün hale geldi
- Bu **switch-case** yapısını daha rahat kullanılır hale getirdi

```
switch(int/String-ifade){  
    case sabite1, sabite2, sabite3:  cümle(ler); break;  
    case sabite4, sabite5:  {cümle(ler); break;}  
    . . .  
    default:  cümle(ler); break;  
}
```

switch-case - VI



- **switch** bloğununun içi boş olabilir yani **case** ve **default** zorunlu değildir ama **switch**'e bir tane argüman geçmek zorunludur ve o argüman **null** olmaz
- **default** değer alamaz ve en fazla bir tane olabilir
- **default** zorunlu değildir ama akışın tamliğini açısından önemlidir:
 - Umulmadık değerlere karşı davranış belirlemek ya da özel durumları yönetmek için **default** kullanılmalıdır

switch-case - VII



- **default** sıra dışı durum (exception) fırlatabilir; bu durumda **break** kullanılamaz
- Sıra dışı durum yönetimi (exception management), daha sonraki Java eğitiminin konusudur
- Hiç **case** olmadan da **default** kullanılabilir
- **default** ve **case** ifadelerinin sırası önemli değildir

switch-case - VIII



- **switch**'e geçilen ifadenin değerine uyan **case** için bir **break** yoksa akış, ilk karşılaşılan **break**'a kadar ya da hiç **break** yoksa **switch** bloğundan çıkana kadar tüm **case**'ler çalıştırılır
- Bu durum *aşağıya kadar inme/akma (fall through)* olarak adlandırılır ve **switch-case**'in kullanımını hataya açık hale getirir
- Çünkü olması gereken bir **break** unutulduğunda **switch** akışı istenmeyen şekilde çalışacaktır

TraditionalSwitchStatement - I



- `decision.switchCase.TraditionalSwitchStatement`
- `switchDemo1()` metodunda şunları yapın ve sonuçları gözlemleyin:
 - Birden fazla `case`'in sabitesini aynı yapın,
 - `case`'lerinde birine sabite yerine `i` gibi bir değişken koyun,
 - `case`'lerinde birine sabite yerine uygun değer döndüren metot çağrısı koyun,

TraditionalSwitchStatement - II



- `decision.switchCase.TraditionalSwitchStatement`
- `switchDemo2 ()` metodunda `break`'in olmamasının sonuçlarını ve zaman zaman olabilecek faydasını da gözlemleyin
- `switchDemo3 ()` metodundaki kullanımın, `switchDemo2 ()` 'daki kullanımdan daha kısa ve şık olduğunu da gözlemleyin
- `switchDemo4 ()`

switch-case - IX



- **switch**'in ifadesi `byte`, `short`, `char`, `int` ya da `String` olmalıdır
- İlkellerin nesne (wrapper, sarmalayan) halleri de olabilir: **Character**, **Byte**, **Short**, **Integer**
 - Sarmalayan tipler ileride ele alınacaktır
- **switch**'in ifadesinde `String` kullanımı Java 7 ile gelmiştir
- **switch**'in ifadesi `long`, `float` ya da `double` olamaz, bu bir derleme hatasıdır

StringInSwitchDemo



- `decision.switchCase.StringInSwitchDemo`

switch-case - X



- Java 21 ile **switch**'in, geçilen bir referansın gerçek tipini öğrenmek için kullanımı mümkün olmuştur
- Buna **pattern matching** de denir
- İleride ele alınacaktır

Yeni switch-case Yapıları - I



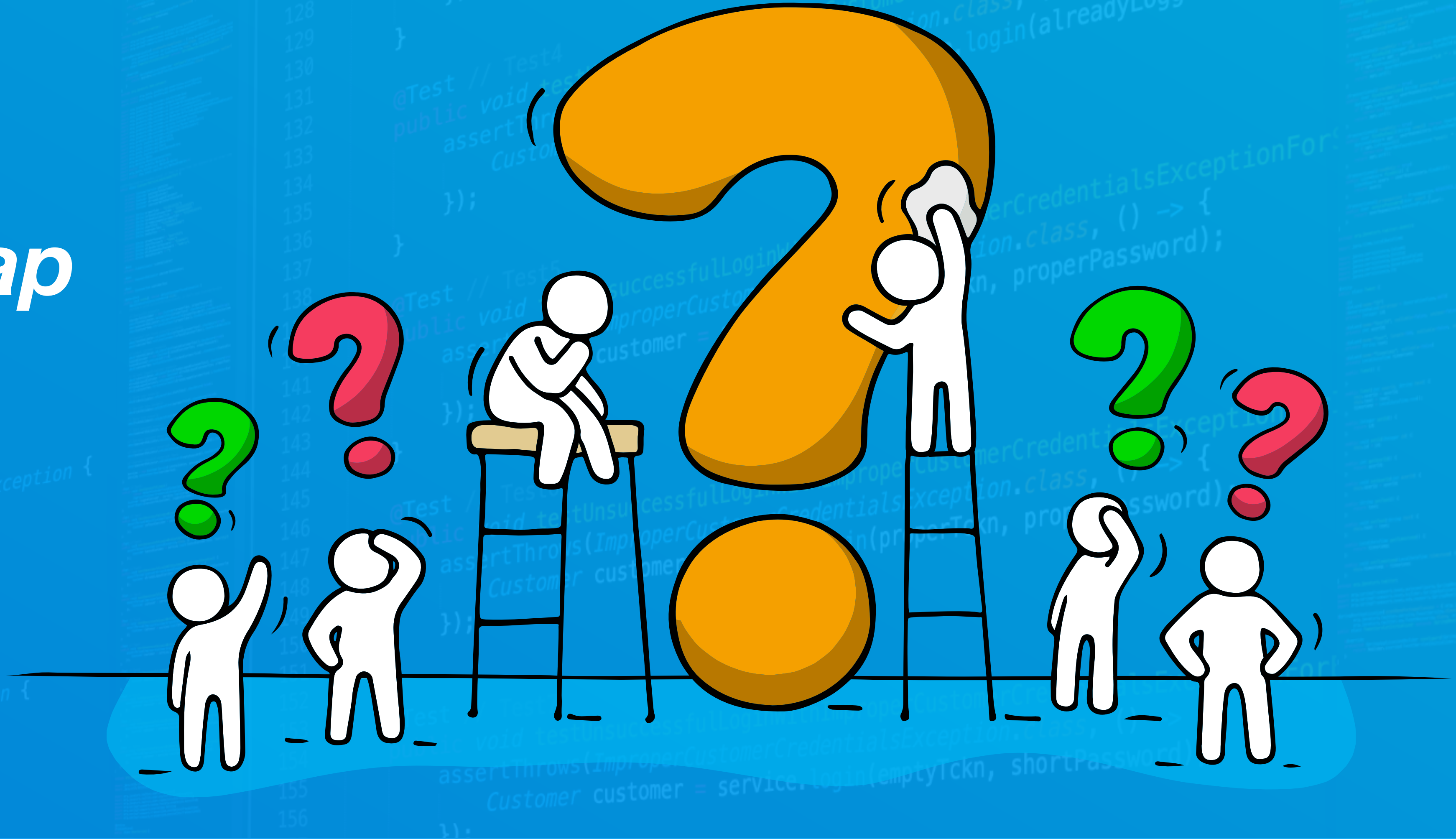
- Java'da ilk sürümünden bu yana var olan geleneksel şekle eklenen, 12. ve 14. sürüm ile gelen yeniliklerle birlikte **switch-case** yapısının dört farklı şekli oluşmuştur
- Bu dört şeklin ikisi cümle (statement) ikisi ifadedir (expression)
 - Unutmayın, cümle değer üretmez ama ifade değer üretir

Yeni switch-case Yapıları - II



- Bu dört şekil şunlardır:
 - `break` ile kullanılan geleneksel **switch-case** cümlesi
 - Bu şekil az önce ele alındı
 - `->` ile kullanılan, `break`'e ihtiyaç duyurmayan **switch-case** cümlesi
 - `->` ile kullanılan **switch-case** ifadesi (expression)
 - `yield` ile kullanılan **switch-case** ifadesi (expression)

Soru ve Cevap Zamanı!





switch-case

-> ile Cümle

switch-case -> ile Cümle — I



- **switch-case**'in ilk halinde **break**'in getirdiği hataya açık durum, bu halin kullanımını zorlaştırmaktadır
- **switch**'e geçilen ifadenin değerine uyan **case** için bir **break** yoksa akış, ilk karşılaşılan **break**'a kadar ya da hiç **break** yoksa **switch** bloğundan çıkana kadar tüm **case**'leri çalıştırır (fall through)
- Bu durumu düzeltmek amacıyla Java 14 ile **->** kullanımı gelmiştir

switch-case -> ile Cümle — II



- Yeni yapıda **switch** ifadesine uyan **case** için sadece **->**'den (arrow, ok) sonra gelen cümle ya da cümleler çalıştırılır
- İlk haldeki gibi, **case** ya da **default** için sıfır ya da daha fazla cümle olabilir; bu cümleler bir blok içinde yer alabilir
- Fark, geleneksel yapıdaki **case** sabitesinden sonra gelen **:** yerine **->** kullanılmasıdır

```
switch(int/String-ifade){  
    case sabite1 → cümle(ler);  
    case sabite2 → {cümle(ler);}   
    . . .  
    default → cümle(ler);  
}
```

switch-case -> ile Cümle — II



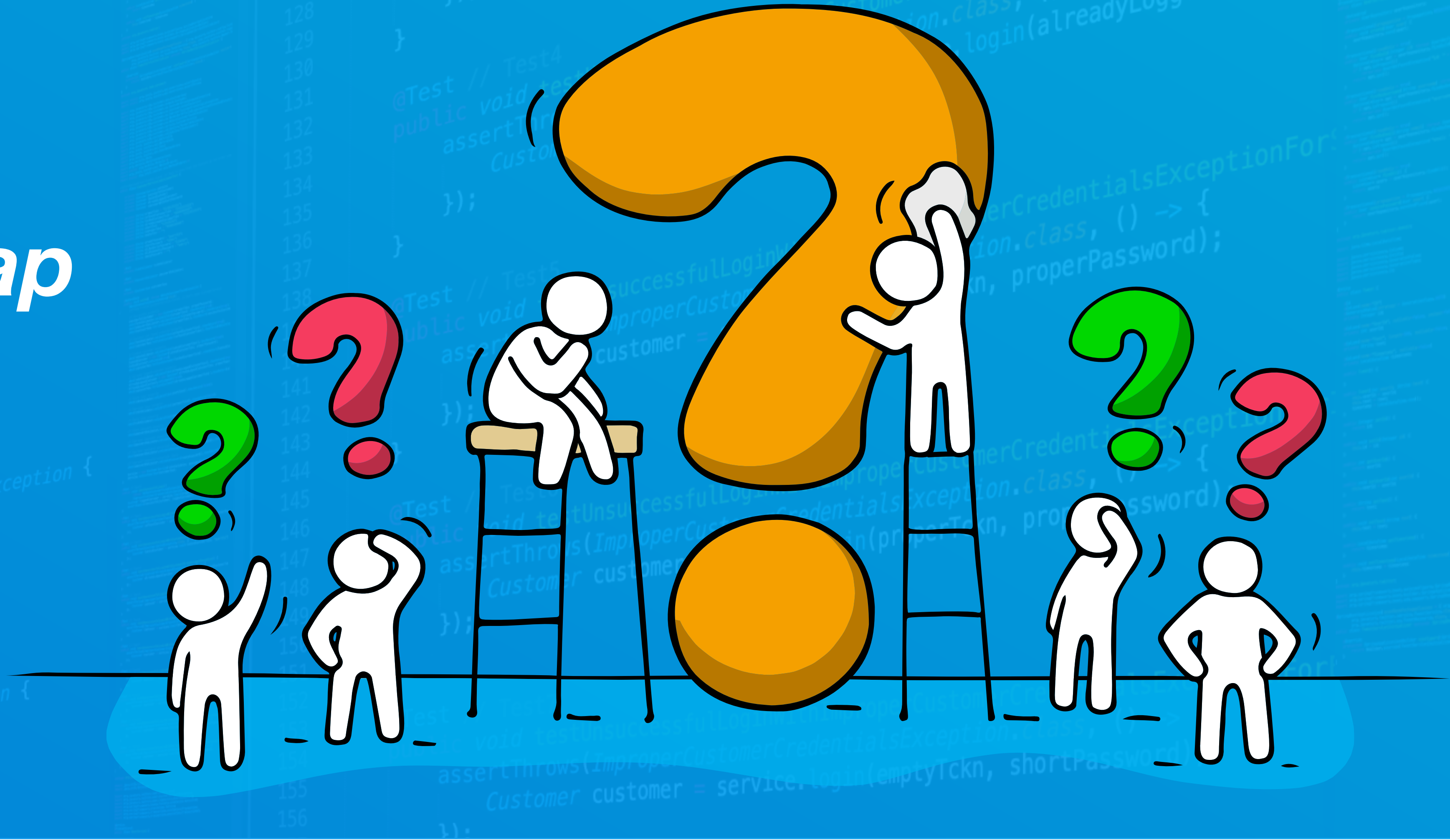
- Aynı **switch** bloğunda hem **:** ile eski yapı hem de **->** ile yeni yapı kullanılamaz
- Bu yeni yapıda **break**'e ihtiyaç yoktur ve **->** ile akışın diğer **case**'lere girmesi önlenir
- **->** varken **break** kullanılamaz, derleme hatası oluşur
- Bundan dolayı **switch-case** yapılarını artık **->** ile kurmak, kodu daha güvenli kılacaktır

NewSwitchStatement



- `decision.switchCase.NewSwitchStatement`
- `switchCaseWithBreakStatement()` ile
`switchCaseWithArrowStatement1()` 'ı kıyaslayın
- `switchCaseWithArrowStatement2()` 'de çoklu değer kullanımını gözlemleyin

Soru ve Cevap Zamanı!





switch-case

-> ile ifade

switch-case -> ile İfade - I



- Java 12 ile birlikte **switch-case** ifadeleri (expression) gelmiş ve bir değer döndürecek şekilde kullanım mümkün olmuştur
- Geleneksel halde **switch-case** bir cümledir (statement), bir değer üretmez
- Bu yeni yapıda **switch** ifadesinin döndüreceği değer **case**'lerden sonra gelen **->** ile belirlenir

```
String monthInString = switch(month){  
    case 1 → "January";  
    case 2 → "February";  
    ...  
    default → "Problem!";  
};
```

switch-case -> ile İfade - II



- **case** sabitesinden sonra -> olduğundan : kullanılmaz
- **switch** ifadesinin değer döndürmesi gerektiğinden artık **default** uygun bir değer döndürecek şekilde zorunludur
- Her **case** ve **default** ifadesi, cümle yapısında olduğu gibi ; ile bitmelidir

```
String monthInString = switch(month){  
    case 1 → "January";  
    case 2 → "February";  
    ...  
    default → "Problem!";  
};
```

switch-case -> ile İfade - II



- Bu yeni yapıda her **case** cümlesi, **switch-case**'in döndürdüğü tipe uygun tipte değer geri döndürmelidir
- **case**'den sonra virgülle ayrılmış birden fazla sabite de kullanılabilir
- Ayrıca artık **switch-case** bir cümle olmaktan çıkıp ifade olduğu için atama ile birlikte bir cümle oluşturur ve sonunda **;** kullanılmalıdır

```
int square = switch(month){  
    case 1 → 1;  
    ...  
    default → -1;  
};
```

```
System.out.println(switch (day) {  
    case 1, 5, 7 → 6;  
    case 2 → 7;  
    case 4, 6 → 8;  
    case 3 → 9;  
    default → -1;  
});
```

switch-case -> ile İfade - IV



- Ayrıca **switch-case**'in döndürdüğü değer alınmak zorundadır aksi taktirde derleyici hatası oluşur

```
switch(month){ // Error!  
    case 1 → 1;  
    ...  
    default → -1;  
};
```

```
int square = switch(month){  
    case 1 → 1;  
    ...  
    default → -1;  
};
```

```
System.out.println(switch (day) {  
    case 1, 5, 7 → 6;  
    case 2 → 7;  
    case 4, 6 → 8;  
    case 3 → 9;  
    default → -1;  
});
```

switch-case -> ile İfade - V



- Yeni yapıda **case**'in kullanımı aşağıdaki gibidir:
- Birden fazla sabite için tek bir değere işaret eden ifadeye sahip olabilir,
- Sıra dışı durum fırlatabilir,
- Bloğa da sahip olabilir

```
System.out.println(switch (day) {  
    case 1, 5, 7 → 6;  
    case 2 → 7;  
    case 4, 6 → 8;  
    case 3 → 9;  
    default → throw new Exception();  
});
```

```
case label_1, label_2, ..., label_n → expression; | throw-statement; | block
```

switch-case -> ile İfade - VI



- **switch**'in bir ifade haline gelmesiyle, geleneksel yöntemde değer üretme işi daha kolay hale gelmiştir
- Geleneksel **switch-case**'de, **switch** bloğu öncesinde tanıtilan bir değişkene değer atamak gerekirken **switch**'in bir ifade haline gelmesiyle bu işlem kolaylaşmıştır

```
int length = 0;
switch (day) {
    case 1, 5, 7 : length = 6; break;
    case 2 : length = 7; break;
    case 4, 6 : length = 8; break;
    case 3 : length = 9; break;
    default: throw new Exception();
};
```

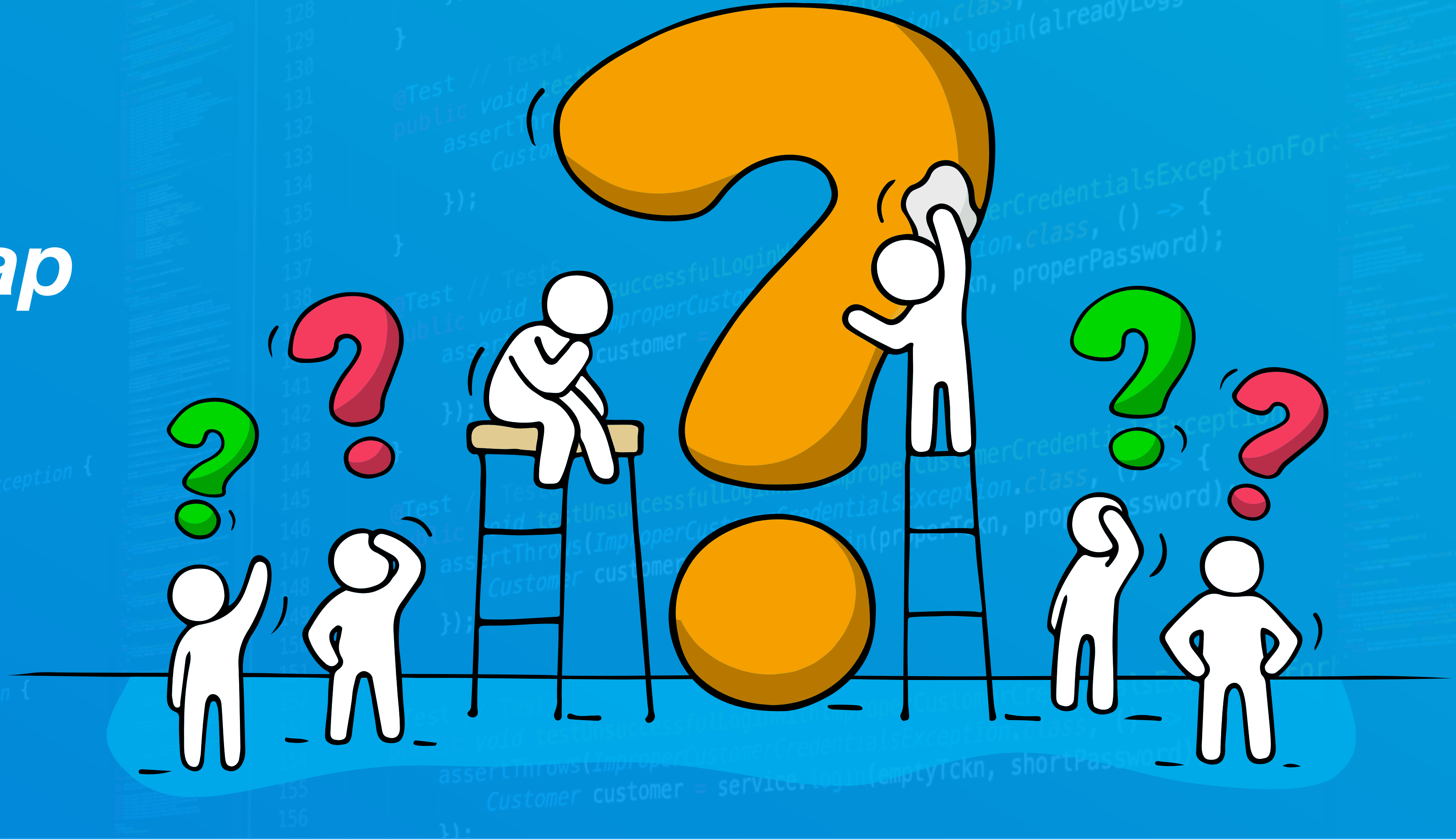
```
int length = switch (day) {
    case 1, 5, 7 → 6;
    case 2 → 7;
    case 4, 6 → 8;
    case 3 → 9;
    default → throw new Exception();
};
```

SwitchExpression



- `decision.switchCase.SwitchExpression`
- `switchWithArrow1()` ve `switchWithArrow2()` daha önceki `switch-case` cümlelerinin ifade halidir
- `switchWithArrow2()` ve `switchWithArrow3()` çoklu değer kullanımına sahiptir

Soru ve Cevap Zamanı!





switch-case

yield ile ifade

switch-case yield ile İfade - I



- Az önce **switch-case** ifadesinde kullanılan **->**'tan sonra blok gelebileceği söylendi
- Fakat **->**'tan sonra blok geldiğinde değerin nasıl döndürüleceği söylenmedi
- **->** bloktan değer döndüremez
- Bu durumda değer döndürmek için **yield** kullanılır

```
System.out.println(switch (month) {  
    case 1 → "January";  
    ...  
});
```

```
System.out.println(switch (month) {  
    case 1 → {"January";}; //  
Error!  
    ...  
});
```

```
System.out.println(switch (month) {  
    case 1 → {yield "January";};  
    ...  
});
```

switch-case yield ile İfade - II



- **yield**, Java 14 ile gelmiş yeni bir ayrılmış kelimedir (reserved word)
- **yield** cümlesi ile içinde bulunduğu **switch** ifadesinden (expression) bir değer döndürülür
- **yield** sadece bir **switch** ifadesinde bulunabilir, **switch** cümlesinde ya da **switch** olmaksızın doğrudan bir metotta vs. bulunamaz, **case** ve sabitelerinden sonra gelir ve muhakkak bir değer döndürür

```
String monthInString = switch(month){  
    case 1 : yield "January";  
    case 2 : yield "February";  
    ...  
    default : yield "Problem";  
};
```

switch-case yield ile İfade - III



- **yield**, **case** sabitesinden sonra hem doğrudan hem de blok içinde kullanılabilir
- Doğrudan kullanıldığında **case** sabitesinden sonra **:** gelir
- **case** sabitesinden sonra **->** varsa bu durumda **yield** ancak blok içinde kullanılabilir

```
System.out.println(switch (month){  
    case 1 → "January";  
    ...  
});
```

```
System.out.println(switch (month) {  
    case 1 → {yield "January";};  
    ...  
});
```

switch-case yield ile İfade - IV



- Dolayısıyla blok olmadığı durumda **->** ve **yield** birbirlerinin yerine kullanılabilir
- Blok içinde kullanıldığı durumda **yield** son cümle olmalıdır
- Bu anlamda **yield**, metottan değer döndüren **return** gibi davranmaktadır

```
System.out.println(switch (month){  
    case 1 → "January";  
    ...  
});
```

```
System.out.println(switch (month){  
    case 1 : yield "January";  
    ...  
});
```

```
System.out.println(switch (month) {  
    case 1 → {yield "January";};  
    ...  
});
```

switch-case yield ile İfade - V



- **yield** kullanıldığında **case**'e bağlı olarak değer döndürüldüğünden **break** kullanılamaz
- **yield** `null` döndürebilir ama sıra dışı durum fırlatamaz
- Sıra dışı durum, **yield** kullanılmaksızın, doğrudan fırlatılmalıdır

```
System.out.println(switch (day) {  
    case 1, 5, 7: yield 6;  
    case 2: yield 7;  
    case 4, 6: yield 8;  
    case 3: throw new Exception();  
    ...  
});
```

SwitchExpression



- `decision.switchCase.SwitchExpression`
- `switchWithYield1()`, `yield`'ın `case` sabitesinden sonra doğrudan, `switchWithYield2()` ise blok içinde kullanımını göstermektedir
- `switchWithYield3()`, `yield`'ın en geniş kullanımını göstermektedir

yield Bağlamsal Anahtar Kelimesi



- **yield** bir bağlamsal anahtar kelimedir (contextual keyword)
- Dolayısıyla bazı bağlamlarda belirteç (identifier) yani isim olarak kullanılabilir ama yine de bazı kısıtlara tabidir:
- **yield** tip ismi (type identifier) olarak kullanılamaz,
- **yield** bir niteleyiciye (qualifier) sahip olmadan metot ismi olarak da kullanılamaz
- Ama değişken ismi olarak kullanılabilir

YieldContextualKeyword



- `decision.switchCase.YieldContextualKeyword`
- `yield`'ın metot ismi (identifier) olarak kullanılmasının nasıl problemlere yol açtığını gözlemleyin



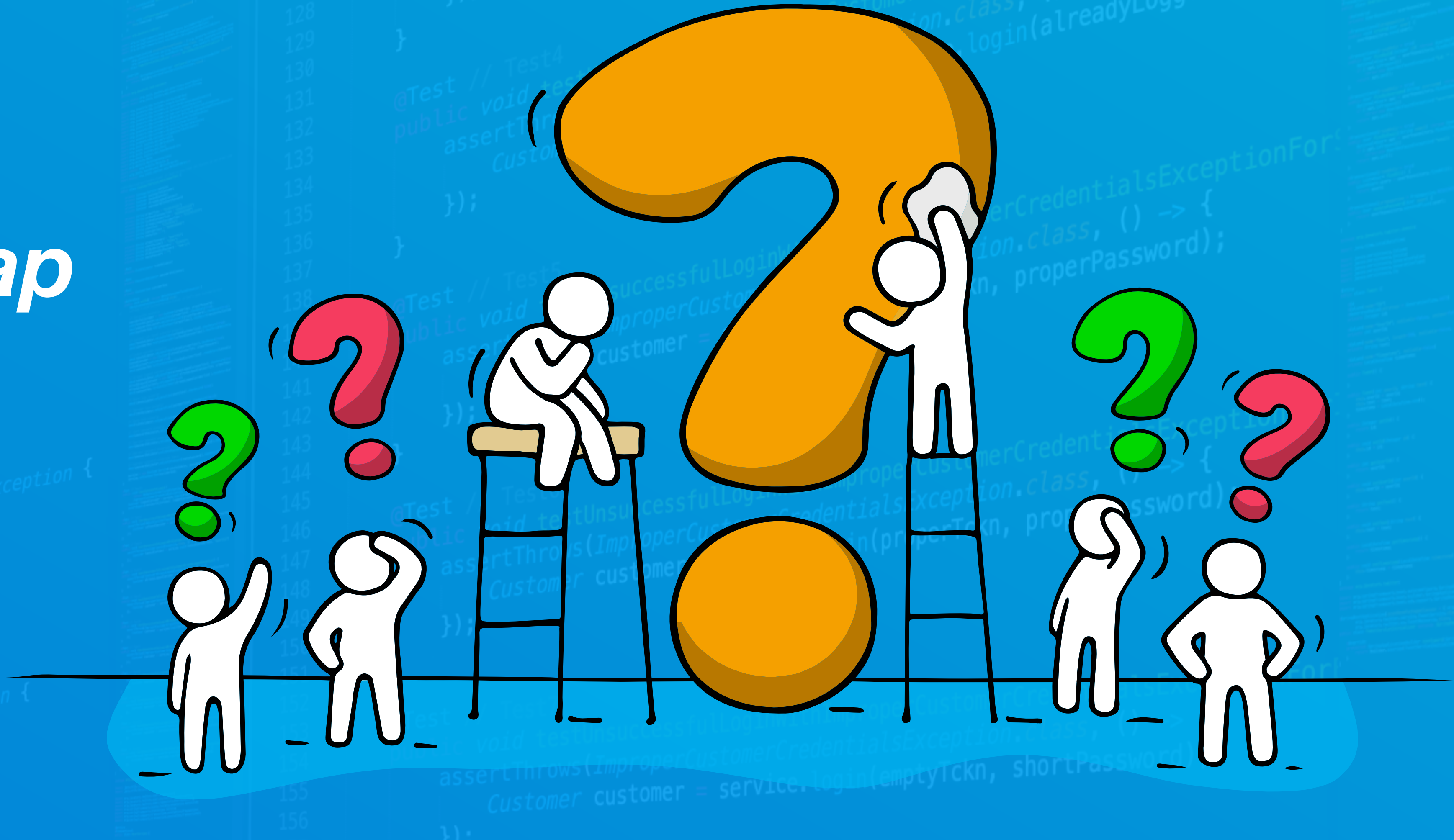
- Bir önceki uygulamayı `switch` ifadeleri ile önce `->` sonra da `yield` kullanarak yapın
- `case` sabitesinden sonra `yield`'ı hem doğrudan hem de blok içinde kullanın

switch-case ve if



- Görüldüğü gibi **switch-case**'in **if-else**'ten farkı, ilkinin önceden belirlenmiş durumlar üzerinden seçme yaparken ikincisinin koşul ifadelerine göre karar vermesidir
- Aslen seçme (selection) de ancak karar (decision) ile yapılabilir ama **switch-case**'in yapısı onu karar vermenin bir özel hali olan seçme için kullanılmasını sağlar
- Bu farktan dolayı **switch-case** değer üretebilir

Soru ve Cevap Zamanı!





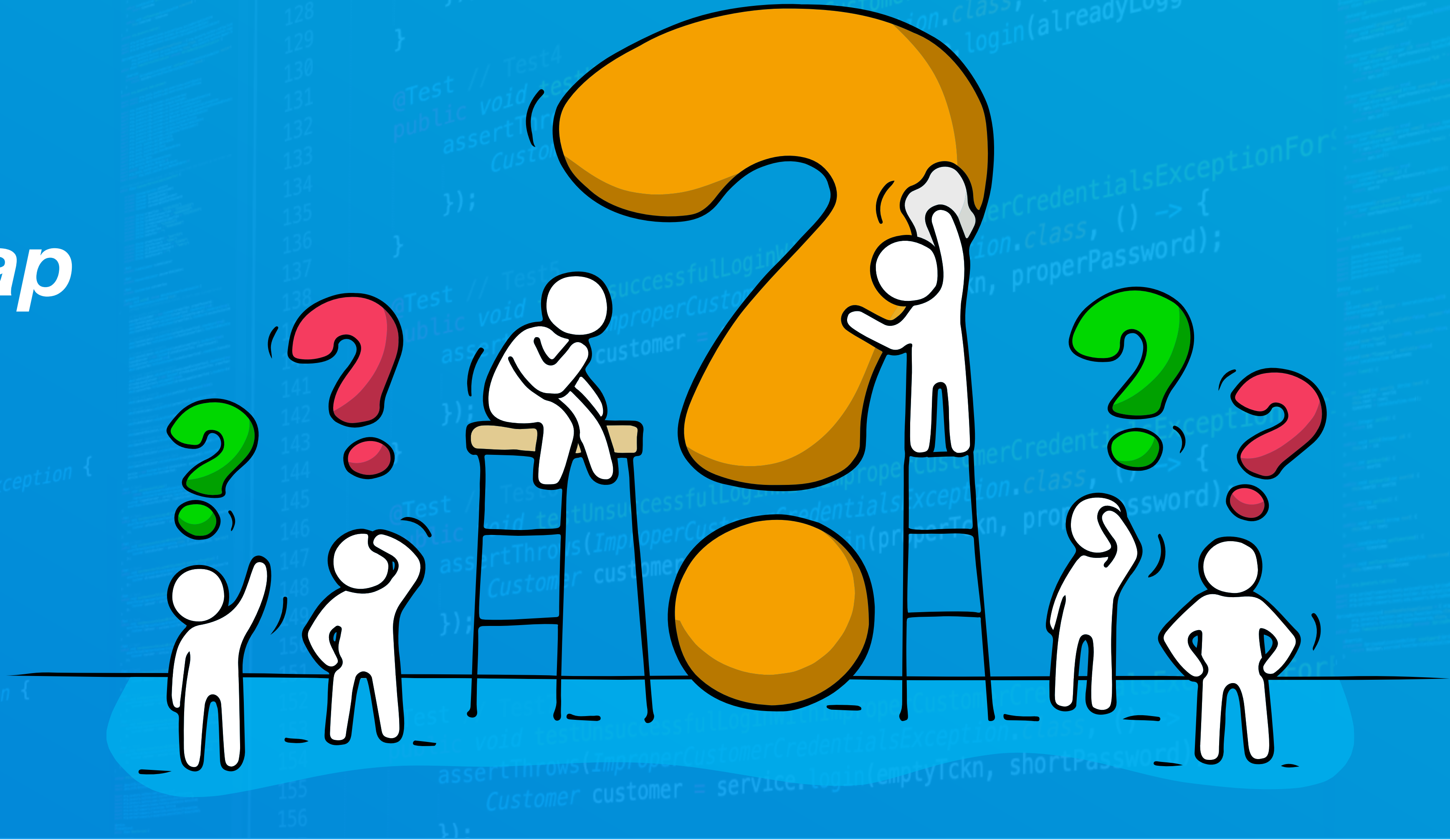
switch-case Pattern Matching

switch-case ile Pattern Matching



- Ayrıca Java 21 ile, **switch-case** kullanarak, geçilen bir referansın gerçek tipini öğrenmek için kullanımı mümkün olmuştur
- Hem pattern matching konusu hem de bunun **switch-case** ile kullanımı ileride ele alınacaktır.

Soru ve Cevap Zamanı!



Diğer Kontrol Yapıları





break ve continue

break ve continue



- **break** ve **continue**, sadece tekrarlı yapılarda kullanılan benzer iki kontrol mekanizmasıdır
 - `for`, `while` ve `do while`
- **break**, `switch` içinde de kullanılır, daha önce görmüştük
- Bu yapılar dışında kullanılırsa derleyici hatası oluşur
- Etiketli ve etiketsiz olmak üzere iki halleri vardır

break



- Etiketsiz halinde **break** içinde bulunduğu tekrar yapısını tamamen kırar ve akış, tekrar bloğunu takip eden cümleden devam eder
- Dolayısıyla **break**, içinde bulunduğu tekrar yapısından tamamen çıkmak için kullanılır
- Daha önce ele aldık, **switch-case** bloğunda **break**'in etiketsiz hali birden çok kere kullanılır ve **switch-case** bloğunu kırar

continue



- Etiketsiz halinde **continue**, içinde bulunduğu tekrar yapısının sadece o anda çalışan adımını sonlandırır ve akış aynı tekrar bloğunun bir sonraki adımından devam eder
- **continue**, **break**'in tersine, içinde bulunduğu tekrar yapısını kırmaz, sadece o adım için sonlandırır
- Ama **break**, içinde bulunduğu tekrar yapısını tamamen bitirir

BreakAndContinueDemo



- `BreakAndContinueDemo`

Etiketli break



- Etiketli halinde **break**'ten sonra bir etiket kullanılır
 - Bu etiket, bir tekrar yapısını göstermelidir aksi halde hata olur
 - Ayrıca **break**, etiketin gösterdiği tekrar yapısının içinde olmalıdır aksi takdirde hata alınır
- Tipik olarak, iç içe tekrar yapılarının olduğu hallerde, **break** en içteki tekrar yapısının içinde kullanılır ve etiket dıştaki bir tekrar yapısını gösterir ve bu şekilde dıştaki, etiketli tekrar kırılır

Etiketli continue



- Etiketli halinde **continue**'dan sonra bir etiket kullanılır
 - Bu etiket, bir tekrar yapısını göstermelidir
 - Bu durumda etiketlenmiş tekrar yapısının o anki adımını sonlandırır ve akış sonraki adımdan devam eder
- Tipik olarak **continue** da, içe içe tekrar yapılarının olduğu hallerde, en içteki tekrar yapısının içinde kullanılır ve etiket dıştaki bir tekrar yapısını gösterir ve bu şekilde dıştaki, etiketli tekrarın o anki adımı sonlandırılır

Etiket (Label)



- Java'da etiket (label), sadece içinde **break** ve **continue** ifadelerinin olduğu tekrar yapılarını işaretlemekte kullanılır
- Etiket olarak herhangi geçerli bir isim kullanılabilir
- İç içe olmamak şartıyla, aynı kapsamda, aynı isim birden fazla etiket olarak kullanılabilir
- İç içe tekrar yapıları varsa etiketleri, tabii olarak farklı olmalıdır

BreakAndContinueWithLabelDemo



- `BreakAndContinueWithLabelDemo`
- Aşağıdaki durumlarda programın nasıl davranacağını bulmaya çalışın ve sonra da deneyin:
 - `break here;`
 - `break;`
 - `continue here;`
 - `continue;`

break ile Durma Problemi Çözümü



- Soldaki koddaki durma problemi, örneğin belli bir sayıda tekrardan sonra **break** ile de çözülebilir

```
double d = Math.random();  
while (d < 0.5)  
    System.out.println(d);
```

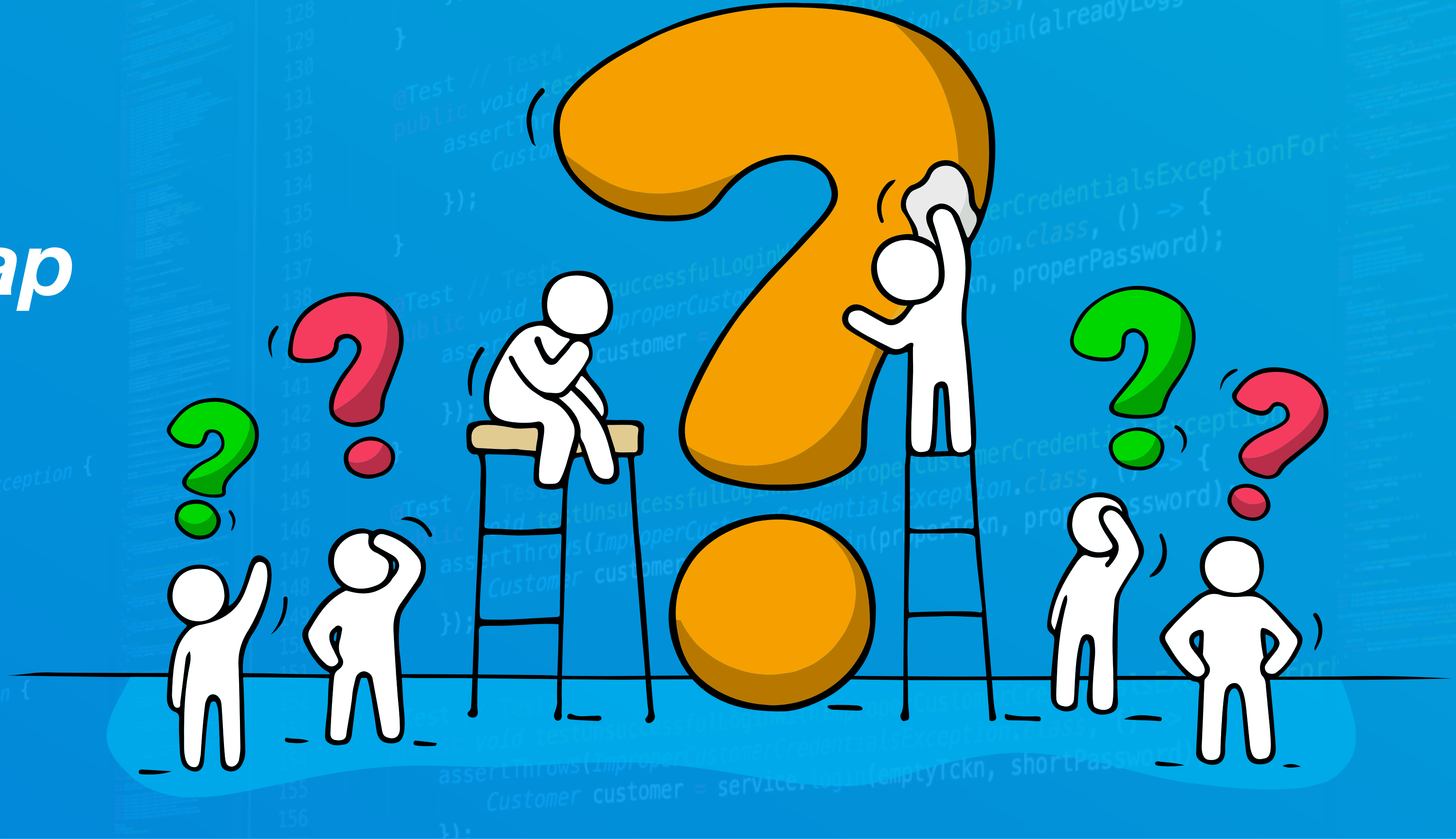
```
double d = Math.random();  
int count = 0;  
while (d < 0.5){  
    System.out.println(d);  
    count++;  
    if (count > n)  
        break;  
}
```

break, continue ve goto



- Dijkstra'nın *Goto Considered Harmful* makalesine rağmen neden Java'da **break** ve **continue** olduğu sorulabilir
- **break** ve **continue**, **goto**'nun eleştirilen kullanımından çok farklıdır
- **goto** ile programda genelde satır numarasıyla istenilen yere atlanabilir (jump)
- **break** ve **continue**'da ise atlama çok sınırlıdır, sadece tekrar yapısı kontrol edilir, dolayısıyla anlamayı zorlaştıran bir tarafı yoktur

Soru ve Cevap Zamanı!





return

return - I



- **return**, bir metottan çağırıldığı ortama dönüş yapmak için kullanılır
- Metot bir değer döndürüyorsa, arayüzünde metot isminden önce dönüş tipi belirtilir ve bu tip, **return** ile döndürülen değerın tipiyle uyumlu olmalıdır

```
double calculateFine(){  
    double fine;  
    ...  
    return fine;  
}
```

```
boolean isEven(int number){  
    if(number %2 == 0)  
        return true;  
    else  
        return false;  
}
```

```
Student createStudent(){  
    Student student;  
    ...  
    return student;  
}
```

return - II



- Metot bir değer döndürmüyorsa **return**'e gerek yoktur ama istenirse dönüş değersiz olarak **return** kullanılabilir
- Bir değer döndürmeyen metotların arayüzünde **void** kullanılır

```
void printFineReport(){  
    ...  
}
```

```
void printFineReport(){  
    ...  
    return;  
}
```

return - III



- **return**'den sonra hiçbir kod olamaz aksi takdirde **erişilemeyen kod (unreachable code)** hatası alınır
- Fakat birden fazla **return** olduğu durumlar, örneğin **if else** kullanarak şartlı değer döndürme, bunun istisnasıdır

ReturnDemo



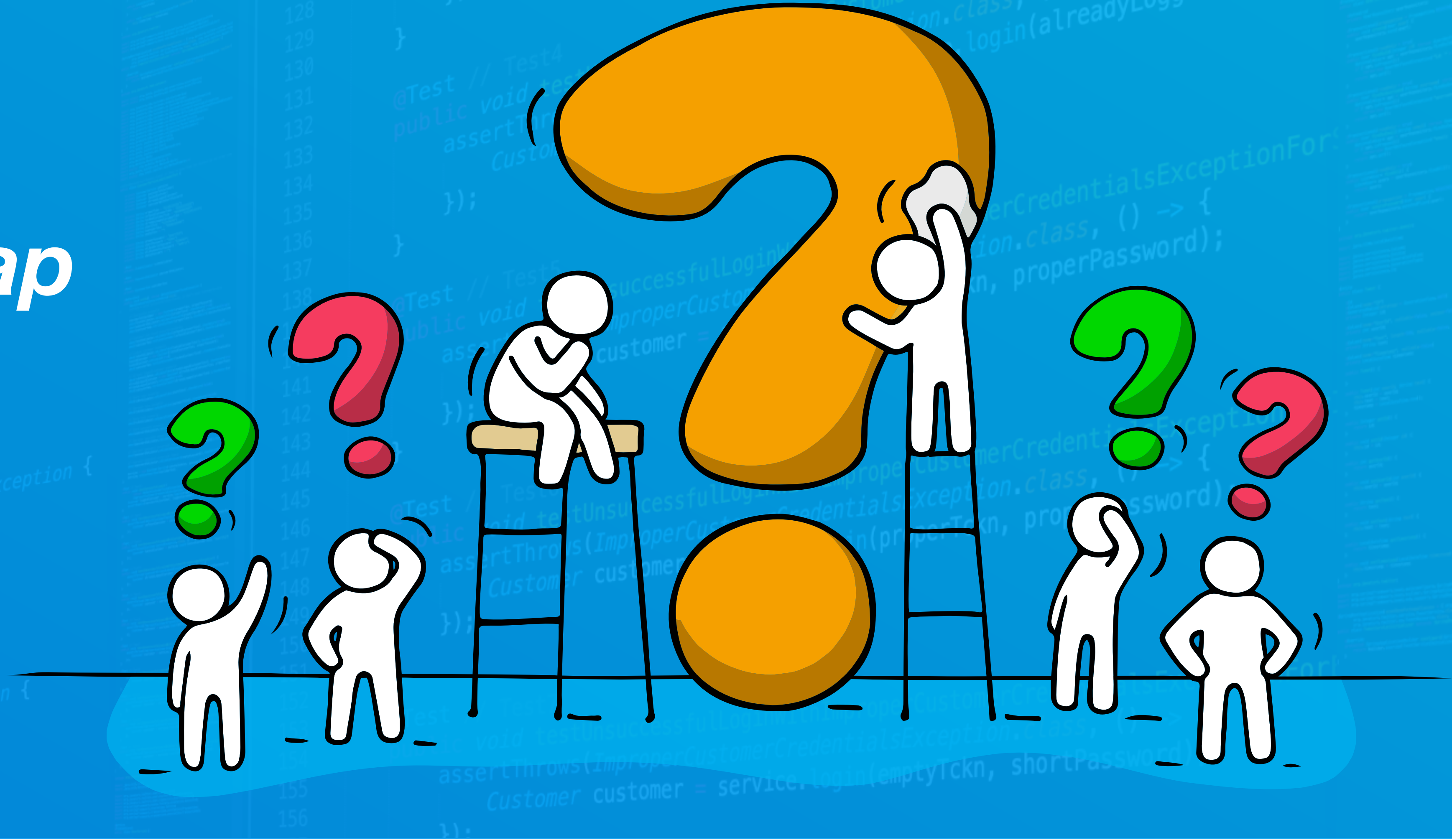
- ReturnDemo

return - IV



- Metotlar ileride ayrıntılı bir şekilde ele alınacaktır

Soru ve Cevap Zamanı!





Diğerleri

Sıradışı Durum Yönetimi



- Java'da **sıradışı durum yönetimi (exception handling)** de akış kontrolü içerir
- İleriki eğitimlerde detaylıca ele alınacaktır

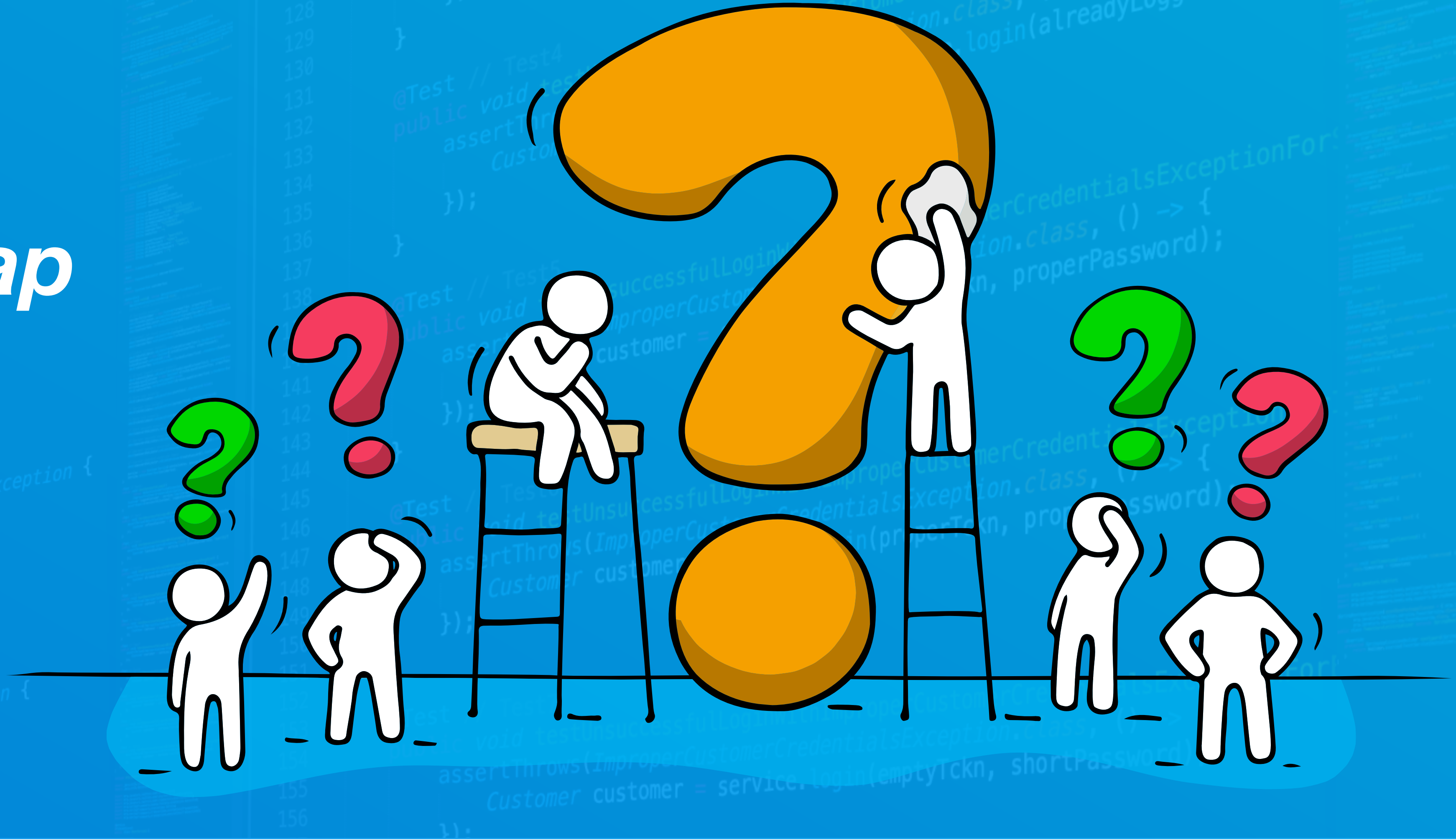
```
try {  
    ...  
} catch (ExceptionType ref) {  
    ...  
} finally {  
    ...  
}
```

goto



- **goto**, Java'da bir kullanımı olmayan ama ileride olabilir diye en başında saklanmış bir kelimedir (reserved word)
- Etiketli **break** ve **continue** ile bu ihtiyaç giderilmiştir

Soru ve Cevap Zamanı!



Özyineleme (Recursion)



Özyineleme (Recursion) - I



- Bir metodun kendini çağırmasına **özyineleme (recursion)** denir
- Özyineleme, tekrar yapılarının bir alternatifi olarak görülebilir
- Yani **for** ya da **while** döngüsüyle yapılacak bir iş, bu döngüler yerine bir fonksiyonun kendisi çağırmasıyla yapılacak şekilde tasarlanabilir

```
void f(){  
    f();  
}
```

Özyineleme (Recursion) - II



- Tabi bu kod sonsuz döngüye girecektir dolayısıyla özyinelemenin duracağı bir nokta olmalıdır ki buna **base case** denir

```
void f(int i, int k){  
    f(i + 1, k);  
}
```

```
void f(int i, int k){  
    if(i == k) // base case  
        return;  
    f(i + 1, k);  
}
```

StackOverflowError



- Bir metodun kendini sürekli olarak çağırması halinde, bir müddet sonra metod çağrılarının yapıldığı stack (yığın) belleği tükenenecektir

```
void f(int i, int k){  
    f(i + 1, k); // StackOverflowError  
}
```

```
void f(int i, int k){  
    if(i == k) // base case  
        return;  
    f(i + 1, k);  
}
```

- Bu durumda JVM **StackOverflowError** hatası ile çalışmasına son verecektir
- Özyinelemeli metod çağrılarında bu durum base case ile önlenmelidir

Recursion



- `loop.recursion.Recursion`
- `StackOverflowError` hatasının nasıl oluştuğuna ve önlenmesine dikkat edin

Özyineleme (Recursion) - III



- Bazı algoritmalar özyinelemeli (recursive) olarak çalışabilir
- Özyinelemeli olarak çalışan algoritmaların, özyinelemesiz yani özyineleme kullanılmadan yazılmaları da söz konusudur
- Özyinelemede, bir problem alt parçalara bölünür ve alt parçaların çözümü ana problemin çözümü olur
- Örneğin verilen bir sayıdan geriye saymak hem klasik döngü ile hem de özyineleme ile yapılabilir

Countdown



- `loop.recursion.Countdown`
- Her metod çağrısında geçilen değerin basıldığına ve bir eksiltilerek aynı metada tekrar geçildiğine dikkat edin
- Bu örnekteki özyinelemenin durduğu base case'e dikkat edin



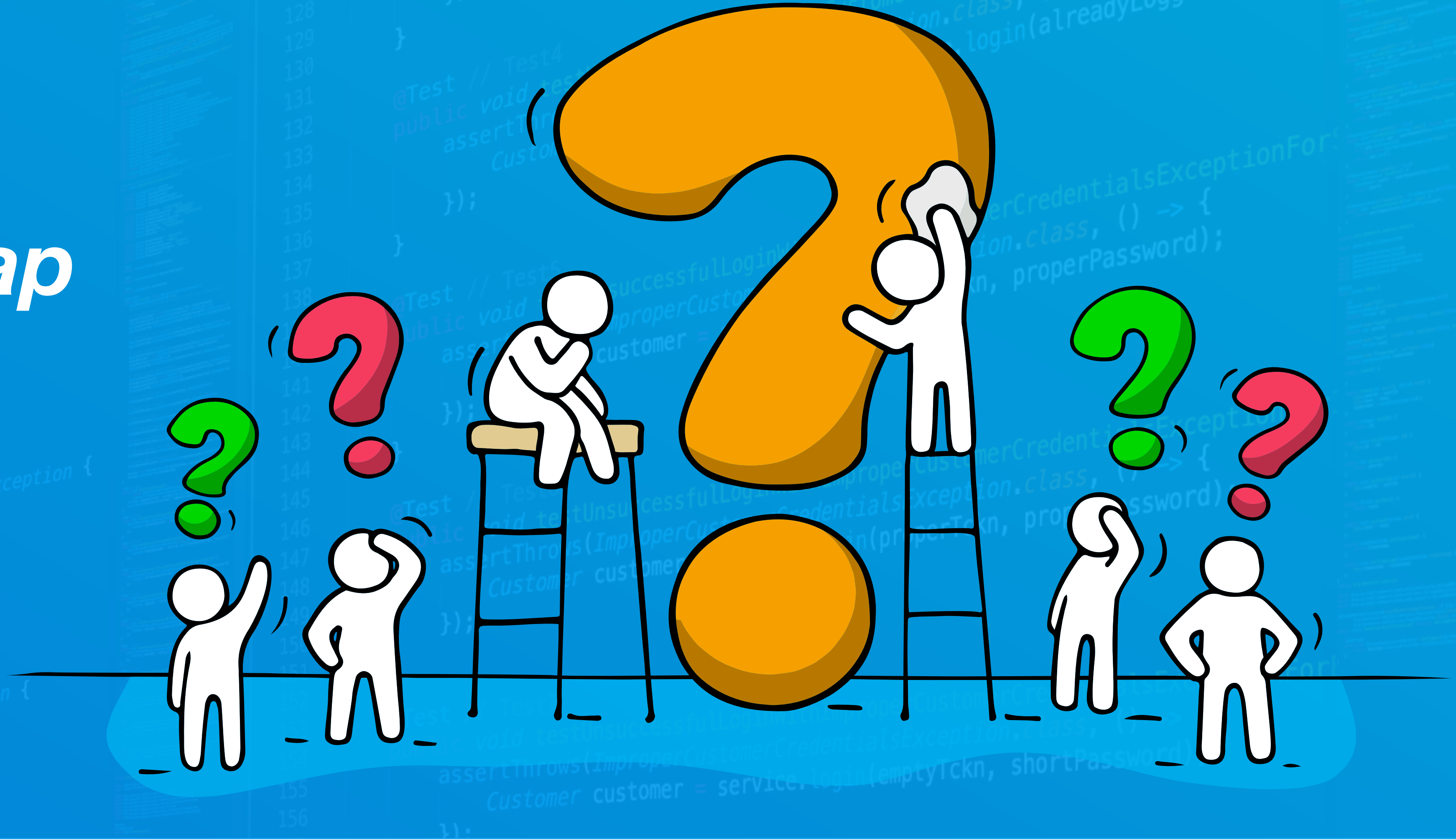
- `loop.recursion.Sum`
- Her metod çağrısında geçilen değerin toplama eklendiğine ve bir eksiltilerek aynı metoda tekrar geçildiğine dikkat edin
- Bu örnekteki özyinelemenin durduğu base case'e dikkat edin

Factorial



- `loop.recursion.Factorial`

Soru ve Cevap Zamanı!



Erişilemeyen ve Ölü Kod



Erişilemeyen Kod



- Java'da, **erişilemeyen kod (unreachable code)** bir derleme hatasıdır
- Örneğin bir metotta tek olan **return**'den sonra kod olmamalıdır
- Sonsuz döngüler de zaman zaman erişilemeyen koda sebep olabilir

```
while(true)
    System.out.println("Selam");
int i = 5;           // Unreachable code!

while(false)
    System.out.println(":)"); //Unreachable code!
```

UnreachableCodeDemo



- `UnreachableCodeDemo`

Ölü Kod - I



- **Ölü kod (dead code)**, akışın ulaşabildiği ama programın davranışını değiştirmeyen koddur
- Erişilemeyen kodun aksine derleme hatası değildir: derlenir, çalışır, hiçbir şeye etki etmez
- Derleyici uyarmaz; ölü kodu IDE'ler ve statik analiz araçları bulur

Ölü Kod - II



- Tipik biçimleri: okunmadan üzerine yazılan atama, hiç çağrılmayan metot, çalışma zamanında hiç girilmeyen döngü, vb. kodlardır

```
int total = 100;    // ölü atama: okunmadan üzerine yazılıyor
total = 20 + 30;
System.out.println(total); // 100 değil 50 basar

for (int k = 8; k < 8; k++) // gövdeye hiç girilmez ama hata da değil
    System.out.println(k);
```

```
if (false)        // Derlenir! Gövde ölü koddur
    System.out.println("x");

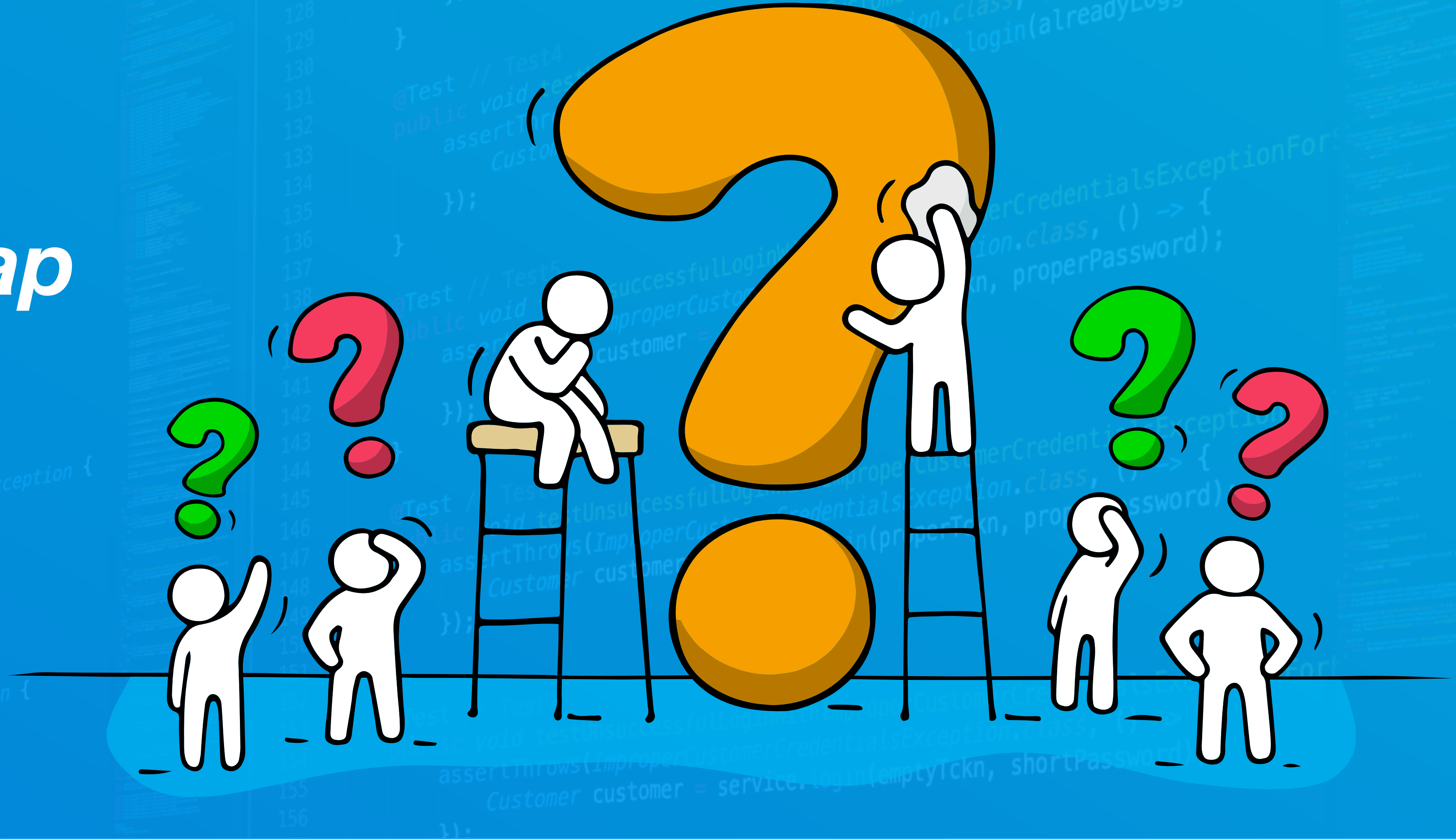
while (false)     // Derleme hatası: unreachable statement
    System.out.println("x");
```

DeadCodeDemo



- DeadCodeDemo

Soru ve Cevap Zamanı!



Blok Kullananmak?



Cümle Ya da Blok - I



- Kontrol yapılarında blok kullanılmazsa tipik olarak tek bir cümle olur ve o cümle de ; ile biter

```
if (mantıksal ifade)  
    cümle
```

- Esasen cümle zorunlu değildir; hiç cümle olmadan da pekala kontrol yapısı yazılabilir ama bu yüksek ihtimalle bir hatadır

```
for (; );
```

```
if (mantıksal ifade);
```

```
while (mantıksal ifade);
```

Cümle Ya da Blok - II



- Eğer blok kullanılmıyorsa ; yanlışlıkla konabilir ve bu durumda aslında karar ya da tekrar yapısının kontrol ettiği cümle, bu yapıdan bağımsız hale gelir ki bu da bir hatadır
- Benzer şekilde bloksuz tek satırlı kontrol yapısına yeni bir cümle eklemek de hataya yol açabilir
- Böyle hatalara düşmemek için blok kullanmak gereklidir

```
for (; ;)  
  A statement // Amaçlanan
```

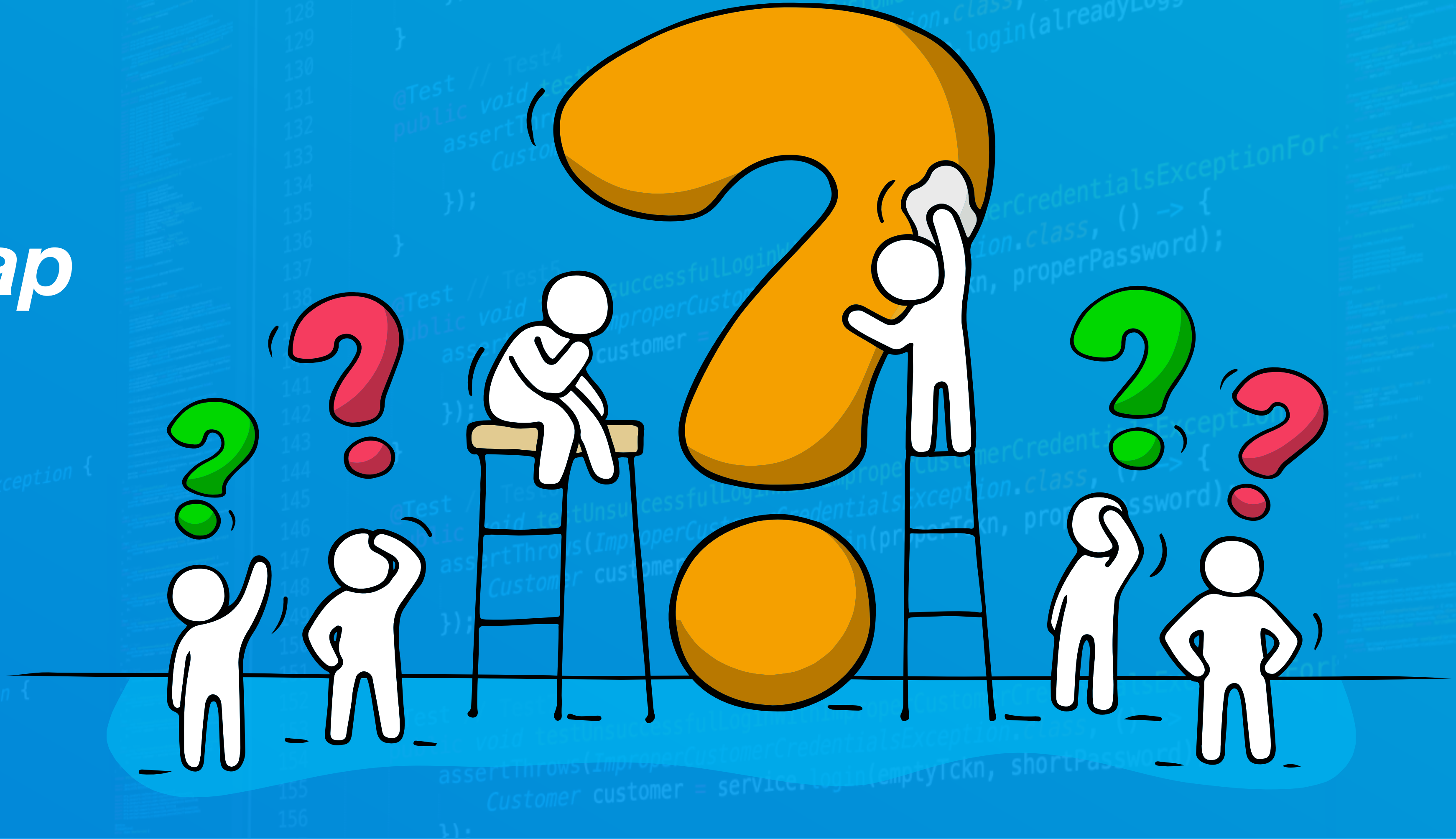
```
for (; ;)  
  A statement // İlk hali
```

```
for (; ;);  
  A statement // Problem!
```

```
for (; ;)  
  A statement  
  Another statement // Problem!
```

```
for (; ;){  
  A statement // Çok daha iyi  
}
```

Soru ve Cevap Zamanı!



Ödevler





1. Yandaki kod derlenir mi?
 - a. Derlenmezse hata nedir, nasıl düzeltirsiniz?
 - b. Derler ve çalışırsa basılacak değerleri tahmin edin
 - c. Sonra yazıp sonucu gözlemleyin

```
public class SwitchScope {  
    void main() {  
        int x = 2;  
  
        switch(x) {  
            case 1:  
                int y = 10;  
                System.out.println(y);  
                break;  
            case 2:  
                y = 20;  
                System.out.println(y);  
                break;  
            case 3:  
                int y = 30;  
                System.out.println(x);  
                break;  
        }  
    }  
}
```



2. Aşağıda ? : işlemcisinin 3 defa kullanımıyla yazılmış tek satırlık kodu **if-else** ile daha okunur hale getirin.

```
cardInfo.setIdNo(cardInfo.getTcCitizen() ?  
    (formUtils.isNullOrEmptyString(verify0tpWSResponse.getTckn()) ?  
        registrationForm.getTckn() : verify0tpWSResponse.getTckn()):  
    (formUtils.isNullOrEmptyString(verify0tpWSResponse.getCustno())) ?  
        registrationForm.getTckn(): verify0tpWSResponse.getCustno());
```



3. Bir tam sayının bölenlerini bulan bir program yazın
4. Girilen bir sayının asal sayı olup olmadığını belirleyen bir program yazın
5. Girilen bir sayıya kadar kaç tane asal sayı olduğunu hesaplayan bir program yazın
6. 2. dereceden $ax^2+bx+c = 0$ şeklindeki bir denklemin köklerini hesaplayan bir program yazın



7. FizzBuzz oyununu yazın. Bilgi için https://en.wikipedia.org/wiki/Fizz_buzz



8. `hw.ch10.Bottles` sınıfının nasıl çalıştığını inceleyin

```
class Bottles {
    static void printSong(Object stuff, int n) {
        String plural = (n == 1) ? "" : "s";
        loop: while (true) {
            System.out.println(n + " bottle" + plural + " of " + stuff + " on the wall,");
            System.out.println(n + " bottle" + plural + " of " + stuff + ";");
            System.out.println("You take one down " + "and pass it around:");
            --n;
            plural = (n == 1) ? "" : "s";
            if (n == 0) break loop;
            System.out.println(n + " bottle" + plural + " of " + stuff + " on the wall!\n");
        }
        System.out.println("No bottles of " + stuff + " on the wall!");
    }
    public static void main(String[] args) {
        printSong("slime", 3);
    }
}
```



9. 1, 12 ve 25 sayfa fotokopinin fiyatları aşağıda verilmiştir. Buna göre çekilecek n sayfalık fotokopinin minimum fiyatını `int fiyatHesapla(int n)` arayüzüne sahip bir metod ile hesaplayın:

- 1 kopya: 5 Kuruş
- 12 kopya: 50 Kuruş
- 25 kopya: 1 Lira

Aynı programı `int özyinelemeliFiyatHesapla(int n)` şeklinde bir arayüze sahip özyinelemeli (recursive) bir metotla tekrar yazın



10. Leibniz'in formülüyle Pi sayısına yakınsayan bir program yazın

11. Bir tam sayının asal bölenlerini bulan bir program yazınız.

Fundamental theorem of arithmetic (the unique factorization theorem or the unique-prime-factorization theorem): Every integer greater than 1 either is prime itself or is the product of prime numbers, and that this product is unique, up to the order of the factors.

https://en.wikipedia.org/wiki/Fundamental_theorem_of_arithmetic

$$100 = 2^2 * 5^2, 1000 = 2^3 * 5^3, 17248 = 2^5 * 7^2 * 11$$



12. Java'nın `java.util.ArrayList` sınıfı, dinamik bir şekilde büyüyüp küçülebilen dizi olarak düşünülebilir. Bir `ArrayList` nesnesi oluşturulduğunda varsayılan kapasitesi 10'dur yani 10 tane elemanı tutabilir. Nesneye elemanlar eklenip kapasite tükenince yeni elemanları tutabilmesi için kapasitesinin artması gerekir. Yeni kapasite, Java 6 ve öncesinde eski kapasitenin 3 katının yarısının bir fazlası, Java 7 ve sonrasında ise eski kapasitenin yarısı kadar eklenmesiyle bulunmaktadır.



- a. 10 varsayılan kapasiteyle başlayan bir **ArrayList** nesnesinin kapasitesinin artışı 1000'e kadar, Java 6 ve öncesiyle Java 7 ve sonrası için adım adım hesaplayın ve yazdırın
- b. Java 7 ve sonrası için de aynı hesabı yapın ama kapasitenin yarısını sağa kaydırma işlemcisiyle bulun



13. Fibonacci sayılarını bulan algoritmaları özyinelemeli ve özyinelemesiz olarak yazın
14. Daha önce çalıştırdığımız ama detayına girmedığımız Monte Carlo simülasyonu ile π sayısına yakınsayan bir programı inceleyin
 - Algoritma için şu sayfadan yararlanabilirsiniz: https://en.wikipedia.org/wiki/Monte_Carlo_method



15. Verilen iki pozitif tamsayının en büyük ortak bölenini (EBOB) bulan programı yazın

- https://tr.wikipedia.org/wiki/Ortak_b%C3%B6len

Bundan Sonra?



Bundan Sonra?



- Bundan sonra dizileri (arrays) ele alacağız

Bölüm Sonu

Soru ve Cevap Zamanı!

